The Definitive Reference: DOM Element Querying Performance in Frontend JavaScript

Audience: JavaScript library and framework authors. **Scope:** the complete native DOM element-querying surface (24 APIs) — specification semantics, engine internals (Blink, WebKit, Gecko), original benchmark measurements, historical/cross-browser evidence, decision rules, and library-author patterns. **Companion sandbox:** `dom-bench/` — self-contained, rerunnable benchmark harness (see Chapter 7 for methodology, launch configuration, and rerun instructions). **Measurement platform:** Chrome 150.0.7871.181 (Blink + V8), new headless mode, Linux x86_64, driven by Puppeteer-core; medians of 10 samples across two full suite runs. Ratios are the signal; absolute numbers are machine-specific. **Research and access date:** 2026-08-12.

1. The Mental Model: Where the Time Actually Goes

Every unit of time a DOM query costs is spent in one of four places: crossing from JavaScript into the engine's DOM implementation, turning a string into a parsed selector, walking (or not walking) the tree, and allocating the objects handed back to script. This chapter builds that cost model from specification semantics — what each API is *required* to do; engine internals (Chapter 2) and measured numbers (Chapter 3) fill in the constants.

1.1 The complete API inventory (24 APIs)

1.1.1 Full inventory table: API | return type | live/static | scope | shadow-aware | O-class

The platform exposes 24 distinct lookup/traversal surfaces, each differing in return type, liveness, scope, or shadow behavior.

API	Returns	Live / static	Scope	Shadow-aware	O-class
document.getElementById(id)	Element | null	n/a (single ref)	Whole document tree	Per-tree only (also on ShadowRoot)	~O(1) hash lookup
getElementsByClassName(names)	HTMLCollection	**Live**	Descendants of receiver	No boundary crossing	Lazy view; indexed re-query per access
getElementsByTagName(name) / …TagNameNS(ns, name)	HTMLCollection	**Live**	Descendants of receiver	No	Lazy view; tag-indexed
document.getElementsByN	NodeList	**Live**	Document only	No	Lazy view; same-object

API	Returns	Live / static	Scope	Shadow-aware	O-class
ame(name)					caching permitted
ParentNode.querySelector(sel)	Element | null	Static result	Descendants of receiver's root	Inside a shadow root, never across	O(candidates), early-exit on first match
ParentNode.querySelectorAll(sel)	NodeList	**Static**	Descendants of receiver's root	Same	O(candidates × selector cost) + full allocation
Element.matches(sel)	boolean	n/a	The element itself, against its root	Works inside shadow trees	O(selector) single-element match
Element.closest(sel)	Element | null	n/a	Self-inclusive ancestor walk	Stops at shadow boundary	O(depth × selector)
Node.childNodes	NodeList	**Live**	Direct children (incl. text)	Light DOM only	O(1) view; per-access validation
ParentNode.children	HTMLCollection	**Live**	Direct element children	Light DOM only	O(1) view; per-access validation
firstElementChild / lastElementChild / nextElementSibling / previousElementSibling / parentElement / childElementCount	Element? / number	Live properties, per access	Local navigation	No boundary crossing	O(1)–O(siblings skipped) per access
Legacy collections (document.forms/images/links/anchors/scripts,	HTMLCollection	**Live**	Document or owning element	No	Lazy views

API	Returns	Live / static	Scope	Shadow-aware	O-class
form.elements, select.options, table.rows, row.cells …)					
document.all	HTMLAllCollection	Live (legacy quirk)	Whole document	No	Lazy view; avoid
window[name] (named access)	Element | WindowProxy | HTMLCollection	Implicit query per access	Matching id / named elements	No	Hidden search on every property read
document.createTreeWalker(…)	TreeWalker cursor	Position pointer, **not** mutation-adjusted	One subtree	Cannot enter shadow roots	O(1) amortized per step; no result allocation
document.createNodeIterator(…)	NodeIterator	**Mutation-adjusted** (pre-remove steps)	One subtree	Cannot enter shadow roots	O(1) amortized per step
document.evaluate(…) (XPath 1.0)	XPathResult	Snapshot static; iterator invalidated by mutation	Context-node subtree	No	O(subtree) + result allocation
document.elementFromPoint(x, y) / elementsFromPoint(x, y)	Element? / Element[]	n/a (hit test at call time)	Viewport hit test	Retargeted to host	O(hit test), layout-dependent
document.caretPositionFromPoint(x, y, {shadowRoots})	CaretPosition?	n/a	Document; opt-in shadow piercing	Only via explicit {shadowRoots}	O(hit test)
DocumentOrShadowRoot.activeElement	Element?	n/a (live property)	Focused element	Host-retargeted	O(shadow depth) retarget
Node.getRootNode({comp	Node	n/a	Which tree the node	composed:true pierces	O(shadow depth)

API	Returns	Live / static	Scope	Shadow-aware	O-class
osed})			lives in	upward	
Event.composedPath()	EventTarget[]	n/a	Full propagation path	Open shadow trees only	O(path length) + array allocation
Event.target	EventTarget?	n/a	Dispatch target	Retargeted outside originating tree	O(1) per retarget step

(Counting …`TagNameNS` and the `*FromPoint` families separately yields 24.) The mixins predict shadow behavior: `DocumentOrShadowRoot` carries ID lookup and `activeElement`; `ParentNode` — implemented by `Document`, `DocumentFragment`, `Element`, `ShadowRoot` — carries `querySelector(All)`, `children`, and the element-child accessors, which is why `shadowRoot.querySelectorAll(...)` works but no document-level call reaches *into* a shadow tree.[1]

1.1.2 The four cost layers of every DOM query

Every query pays some subset of four costs; separating them predicts the winner before benchmarking.

Layer 1 — WebIDL binding crossing. Each property read or method call crosses from the JS heap into the engine's C++ DOM through generated WebIDL bindings: argument conversion, this-value checks, a round trip per getter. Hence `for (i = 0; i < list.length; i++)` is never free even with cached data — `list.length` is a binding call, not a field read. Implicit lookups like `window[name]` are doubly bad: binding crossing *plus* an unrequested query, which the HTML Standard deprecates in favor of explicit `getElementById/querySelector`.[2]

Layer 2 — String atomization and selector parse. Selector-based APIs must *parse a selector* before matching; scope-match throws `SyntaxError` `DOMException` on failure.[1] Parse is pure overhead for one-off calls and the entire differential between `querySelector('#a')` and `getElementById('a')` for a known ID — the latter takes a raw string, never parses, never throws, needs no `CSS.escape()`.[3][4][5] `qS/qSA/matches/closest` all share the throwing parse; validate once, outside hot paths.[5]

Layer 3 — Tree traversal strategy. The dominant term, spec-pinned: Selectors 4's *match a selector against a tree* starts from "a list of *candidate elements*, which are the root elements and all of their descendant elements" and tests each one[6] — `querySelectorAll` is O(subtree size) at minimum, by specification. Against that baseline, `getElementById` is first-in-tree-order by ID[1] (engines: hash map), the `getElementsBy*` family exploits class/tag indexes, and cursor APIs (`TreeWalker`, `NodeIterator`, sibling accessors) make traversal O(1) amortized per

visited node. Every API choice is a choice among indexed lookup, filtered subtree scan, and incremental walk.

Layer 4 — Result allocation. A static `NodeList` is built eagerly — per Zakas' reading of the WebKit sources, "a loop is used to get every result and build up a NodeList," while a live collection is "created by registering its existence in a cache."[7] Every returned `Element` also needs a JS-side wrapper: a 5,000-match `qSA` materializes 5,000 wrappers plus the list; `getElementsByClassName('x')[0]` allocates a view and one wrapper. `TreeWalker` is the extreme — bulk enumeration with no result list at all.[8]

1.2 Live vs static collections — the single most consequential semantic

1.2.1 What is live, what is static

The DOM Standard's default: "A collection can be either *live* or *static*. **Unless otherwise stated, a collection must be live.** If a collection is live, then the attributes and methods on that object must operate on the actual underlying data, not a snapshot of the data." A live collection is a *filter* plus a *root* — a view, not data.[1] Live: all `getElementsBy*`, `Node.childNodes`, `ParentNode.children`, all legacy `HTMLCollections`, `NamedNodeMap`.[9] [10] [11] MDN states the exception flatly: "**The ubiquitous `document.querySelectorAll()` method is the only API that returns a static `NodeList`.**"[9]

1.2.2 Live collection economics

Creation is nearly free; cost is deferred to access: every read of `length`, `[i]`, or `item(i)` re-runs the filter or re-validates a cache that any relevant DOM mutation has dirtied.[1] [7] A static `NodeList` inverts the curve — full walk plus allocation once, then iteration touches plain memory.

	Live (getElementsBy*, childNodes)	Static (querySelectorAll)
Creation	~O(1) — register filter + root[7]	O(subtree) — walk + NodeList + wrappers[7]
Per-access	Re-query or cache re-validation	O(1) memory read
Mutation during iteration	Invalidates cache; length moves under you	No effect — snapshot
for (i < list.length)	Re-validates per iteration; mutation → infinite loop or skips[7]	Safe; fixed length
Wins when	[0]/length-only access, held references, repeated same-query reads	Iteration, mutation-adjacent code, snapshots

Both canonical traps follow from re-evaluating `length` in the loop condition. Growing while iterating never terminates:

```
const lis = ul.getElementsByTagName("li");      // live
for (let i = 0; i < lis.length; i++) {
  ul.appendChild(document.createElement("li"));  // lis.length grows → infinite loop
}
```

The identical code with `querySelectorAll("li")` terminates — the static list never grows.[7] Removing while iterating forward silently skips elements: deleting `live[i]` shifts every later element down one index, so the next read lands past the successor. Fixes: iterate backwards, snapshot once (`Array.from(live)` — MDN's own advice[10]), or remove index `0` until empty. Even without mutation, an uncached `live.length` condition re-validates per iteration; hoist `const n = live.length` or convert once.

1.2.3 When live wins

Live is faster when you exploit the laziness. Single-shot lookups needing only `[0]` or `length` skip result-set construction entirely — `getElementsByClassName('x')[0]` short-circuits relative to building a complete static list, which is why community micro-benchmarks consistently rank it ahead of `querySelectorAll('.x')` here (indicative, engine-dependent: ~5 ms vs ~55 ms over 100k nodes in one reproduced test).[4] [12] Held references amortize Layers 1 and 4: keep one live object instead of re-calling `querySelectorAll` per render; the engine's resolved-set cache serves reads between mutations — the HTML Standard even permits returning the *same object* for repeated `getElementsByName` calls.[11] The trade is symmetric: mutate-heavy code pays invalidation on every change, so static snapshots win whenever the DOM churns during the loop.

1.3 Selector cost ladder and matching direction

1.3.1 Right-to-left matching: every descendant is a candidate

Per-candidate matching is recursive and **right-to-left**: "process it compound selector at a time, in right-to-left order … If any simple selectors in the rightmost compound selector does not match the element, return failure. … Otherwise, consider all possible elements that could be related to this element by the rightmost combinator."[6] The rightmost compound — the *key selector* — filters every candidate; combinators to its left trigger ancestor/sibling walks only for survivors. The key must be your most selective, indexable term; everything left of it is per-survivor verification cost, not candidate reduction.

1.3.2 The cost ladder

Key selector: `#id` (hash lookup) < `.class` ≈ `tag` (class/tag maps) < `[attr]` existence < `[attr=v]` equality < substring operators (`[^=]`, `[$=]`, `[*=]`, `[~=]`, `[|=]` — per-candidate string work, no index) < structural pseudo-classes < `:has()` < `*` (test everything, no index).[6] [13] [14] Structural pseudos split: forward-looking (`:nth-child`, `:first-child`) need sibling

position; backward-looking (`:nth-last-child`, `:nth-last-of-type`, `:only-of-type`) must know what comes *after* the candidate — the browser "must first know everything about all the other elements," far costlier than "matching a selector to an element on the sole basis of its class name."[13] `:has()` extends that lookahead to descendants and siblings; MDN keeps a dedicated optimization note.[14]

Combinators: + (single sibling step) < > (single parent step) < ~ (walk preceding siblings) < descendant (ancestor walk per survivor — worst-case O(n·depth)). Compound chains multiply the per-candidate test: `div.card` costs a type check *and* a class check on every candidate — MDN's over-specificity point.[14] The cascade-side worst case, elements × selector work, "because the browser needs to check each element at least once against every style to see if it matches," bounds one qSA call as well.[13]

1.3.3 `#a .b` is never faster than `.b`

Both selectors face the same candidate set — every descendant of the context. `#a .b` does not shrink it; it *adds*, per `.b`-matching survivor, an ancestor walk hunting `#a`: equal or slower, never faster. Engines prune by `#a` more readily for stylesheet matching than for arbitrary qSA calls — do not rely on it. The only lever the spec's algorithm gives you is shrinking the root: resolve `getElementById('a')` once (hash lookup, no parse) and call `.querySelectorAll('.b')` **on that element**, reducing the candidate set itself. Related trap: `Element.querySelectorAll` matches against the whole tree and filters to descendants, so the left side may match *outside* the receiver — prefix `:scope`.[15] [12] [6]

1.4 Traversal semantics: `closest()`, `matches()`, sibling/child accessors

1.4.1 `closest()` — self-inclusive upward walk

The DOM Standard defines it exactly: parse the selector (throwing `SyntaxError` on failure), take "this's **inclusive ancestors** that are elements, **in reverse tree order**," return the first match, else `null`.[1] Self-inclusive, nearest-match-wins, O(depth × selector). A manual loop is equivalent for simple selectors and preferable in ultra-hot paths — it never throws (validate once outside the loop) and allows per-level early exit:

```
let el = start;
while (el && !el.matches(sel)) el = el.parentElement;
```

`matches()` tests the element against its root; complex selectors with combinators are legal.[1] Both stop at the shadow boundary: a shadow-tree node's ancestors terminate at its shadow root, so `closest()` cannot match the host from inside; crossing upward requires an explicit `el.getRootNode().host` hop.[1] [16]

1.4.2 `children` vs `childNodes` vs element-sibling accessors

`childNodes` is a live `NodeList` of *all* children — elements, text, comments; `children` is a live `HTMLCollection` of elements only.[9] [10] Text-node pollution (every whitespace run is a node) is why `firstElementChild` / `nextElementSibling` exist: they skip non-elements

per access. Cost asymmetry: `children[i]` is an indexed access on a live view; a `firstElementChild` + `nextElementSibling` chain is a cursor walk — O(1)-ish per step, no collection object, no `length` re-validation — cheapest for visiting a few known siblings. None of these accessors crosses a shadow boundary; a host's `children` sees only light DOM, slotted content included.[1] [17]

1.4.3 `TreeWalker` vs `NodeIterator`: mutation semantics

Both come from `document.createTreeWalker/createNodeIterator(root, whatToShow, filter)` and traverse exactly one tree — they cannot descend into shadow roots (WHATWG/dom issue #1189 remains open).[1] The decisive difference is mutation behavior. `NodeIterator` keeps a reference node that the DOM Standard's *pre-remove steps* adjust on removal — it stays correct across concurrent mutation.[1] [18] `TreeWalker` merely "represents the nodes of a document subtree and a position within them"; its `currentNode` gets **no** such adjustment, so removing the current node mid-walk strands the cursor on a detached node.[18] [8] Use `TreeWalker` for read-only bulk enumeration (no result allocation, cheapest per step), `NodeIterator` when the walk mutates. XPath's `Document.evaluate` exposes the same duality: iterator result types are *invalidated* by modification, snapshot types static-but-stale.[19]

1.5 Shadow DOM: the hard boundary

1.5.1 No document-level query pierces a shadow tree

Shadow DOM exists to "have the internals of this tree hidden from JavaScript and CSS running in the page."[20] The mechanism is tree scoping: the DOM Standard scopes IDs, collections, and selector matching to a node's *root*, and each shadow tree is a separate tree with its own root — so *none* of `getElementById`, `getElementsBy*`, `querySelector(All)`, XPath, `TreeWalker`, or `NodeIterator` crosses the boundary.[1] `getElementById` lives on the `DocumentOrShadowRoot` mixin: `shadowRoot.getElementById(...)` works, IDs need only be unique *per tree*, and the document's ID map cannot contain shadow content.[1] For any "find element anywhere" helper, correctness therefore requires degenerating from one indexed lookup into a full recursive walk with a `shadowRoot` check per element; the proposal for a `shadowRoots` option on `querySelector` (WHATWG/dom #1422) exists precisely because today's userland workaround — "literally walk every single element" — is "nontrivial, not to mention very inefficient."[21] Design APIs to accept a root and query inside it (`host.shadowRoot.querySelector(sel)`, open roots only); use `getRootNode({composed:true})` to pierce upward.[16]

1.5.2 Retargeting: events, focus, and hit tests

Where queries may not cross, observation APIs cross *by retargeting to the host*. `event.target` seen outside the shadow tree is the host, not the true origin — the DOM Standard applies a `retarget` algorithm at each shadow-root crossing; `event.composedPath()` returns the full path but omits nodes of `closed` roots.[22] [23] [1] `document.activeElement` is shadow-aware by design: focus inside a shadow tree yields "the root element of that tree" — the host.[24]

`elementFromPoint/elementsFromPoint` are retargeted per CSSWG resolution ("look for the highest shadow host of the element and return that instead"); `caretPositionFromPoint(x, y, {shadowRoots:[…]})` is the single API with an explicit opt-in to pierce named roots.[25] [26] Library rules: in delegated handlers, `event.target.closest(sel)` starts from the host — use `event.composedPath()[0]` when the true origin matters and the tree is open; and never assume `document.querySelectorAll('*')` means "all elements" on a page that uses shadow DOM.[17]

2. Engine Internals: Why the Winners Win

Chapter 1 decomposed every DOM query into four cost layers: binding crossing, selector parse, traversal strategy, and result allocation. This chapter shows how each engine implements each layer — and why the winning APIs win for *structural* reasons, not tuning luck. The short version: every engine keeps an incrementally updated, interned-string-keyed hash map for IDs; every engine caches parsed selectors; every engine matches right-to-left; and no engine can optimize away the JS↔C++ boundary.

2.1 Blink (Chrome/Edge): maps, caches, and the 2026 SelectorQuery rewrite

2.1.1 getElementById = O(1): TreeOrderedMap hash map keyed by AtomicString, maintained incrementally

`document.getElementById(id)` lands in `ContainerNode::getElementById` (`third_party/blink/renderer/core/dom/container_node.cc`), which consults the tree-scope map and only falls back to traversal for detached subtrees:[27]

```
Element* ContainerNode::getElementById(const AtomicString& id) const {
  if (id.empty()) { return nullptr; }
  if (IsInTreeScope()) {
    Element* element = GetTreeScope().getElementById(id);
    ...
  }
  // Fall back to traversing our subtree.
  for (Element& element : ElementTraversal::DescendantsOf(*this)) {
... }
}
```

The map is `TreeOrderedMap` (`core/dom/tree_ordered_map.cc`): a `HeapHashMap<AtomicString, Member<MapEntry>>` keyed by the **interned** id string. Each `MapEntry` caches the first matching element pointer plus a count, and materializes a lazily built `ordered_list` only when duplicate ids exist:[28]

```
void TreeOrderedMap::Add(const AtomicString& key, Element& element) {
  Map::AddResult add_result = map_.insert(key,
MakeGarbageCollected<MapEntry>(element));
```

```
  if (add_result.is_new_entry) return;
  entry->element = nullptr;
  entry->count++;
  entry->ordered_list.clear();
}
```

Two properties make this path nearly free at steady state. `AtomicString` stores its hash **inside the string object** and interned strings compare by pointer, so a hot `getElementById("foo")` loop never touches character data after the one-time atomization. And the map is maintained incrementally — elements register/unregister on insertion, removal, and `id` mutation — so the query path performs zero tree traversal.[28] Steady-state cost: one precomputed hash fetch, one or two bucket probes, one pointer dereference.

2.1.2 getElementsByClassName/TagName: cached live collections in NodeListsNodeData; LiveNodeListRegistry invalidation bitmask

`ContainerNode::getElementsByClassName` does not rescan the tree per call. It returns a cached collection object keyed by `(CollectionType, AtomicString)`:[27]

```
HTMLCollection* ContainerNode::getElementsByClassName(const
AtomicString& class_names) {
  return EnsureCachedCollection<ClassCollection>(kClassCollectionType,
class_names);
}
```

The cache lives in `NodeListsNodeData`, a rare-data side table so nodes with no live lists pay nothing; `AddCache` returns the existing `LiveNodeListBase` on a hit.[29] Liveness is achieved by **invalidation**, not re-querying: `ContainerNode::InvalidateNodeListCachesInAncestors` walks ancestors on child or attribute changes, gated by `Document::ShouldInvalidateNodeListCaches(attr_name)`. A global `LiveNodeListRegistry` holds a bitmask of which invalidation types exist *anywhere*, so attribute mutations are free when no live list cares:[30]

```
bool ContainsInvalidationType(NodeListInvalidationType type) const {
  return mask_ & MaskForInvalidationType(type);
}
```

Consequences: repeated `getElementsByClassName("x")` calls return the *same C++ object*; the rescan happens only after a relevant mutation. But every `.length` or indexed access on a dirtied list can trigger a fresh traversal, and mutating `class` while iterating a class-based list invalidates the very list under your feet.[30]

2.1.3 SelectorQueryCache (256 parsed selectors) + 2026 rewritten SelectorQuery (CL 7900254): rightmost-first state machine, ID-accelerated search, TinyBloomFilter subtree skipping, NthIndexCache

The entry point shows the layering:[27]

```
Element* ContainerNode::QuerySelector(const AtomicString& selectors,
ExceptionState& es) {
  SelectorQuery* selector_query =
GetDocument().GetSelectorQueryCache().Add(
      selectors, GetDocument(), es);
  if (!selector_query) return nullptr;
  return selector_query->QueryFirst(*this);
}
```

`SelectorQueryCache` (`core/css/selector_query.cc`) is a per-Document cache of **fully parsed selectors keyed by selector string**, capped at 256 entries with FIFO eviction — repeat an identical string and parsing is skipped entirely.[31]

The 2026 rewrite by Steinar H. Gunderson (reapplied as CL 7900254 after one revert) replaced per-element interpretation with a compiled form: `SelectorQuery::BuildCompounds` decomposes the selector into `Compounds` (id / tag / class / exact-attribute / `:nth-child` facts each, `std::reversed` so the **rightmost/subject compound matches first**), and `Execute` runs a small state machine over the DOM.[31] [32] Two accelerations matter most:

1. **ID-accelerated search.** If any remaining compound contains an ID selector, Blink reuses *the same TreeOrderedMap as getElementById* to jump straight to the candidate subtree — `#form .field` starts at the `#form` element, not at the document root:[31]

```
// Now go and see if any of the remaining compounds contain an ID
selector.
// The DOM maintains special structures to locate IDs quickly, so this
is
// our preferred acceleration if available;
...
} else {
  match(scope.getElementById(id));
  return;
}
```

2. **Bloom-filter subtree skipping.** Each compound carries an `Element::TinyBloomFilter` of identifier hashes (the same `SelectorFilter` technology as style resolution); whole subtrees that provably cannot match are rejected with one AND+compare:[31]

```
if (IsA<Document>(root_node) && !
To<Document>(root_node).CouldMatchFilter(selector_filter)) {
  // Neither this nor its children could match this query, so we can
```

```
exit early.
  return false;
}
```

Anything the compound fast path cannot prove (namespaces, most pseudo-classes, combinator precision) sets `need_full_check_` and is re-verified with the general `SelectorChecker`; `:nth-child(an+b)` and `:first-child` *are* fast-pathed via a per-query `NthIndexCache`. `QueryFirst` bails at the first hit (`kShouldOnlyMatchFirstElement`) — structurally, `querySelector` is cheaper than `querySelectorAll(...)[0]`.[31]

2.2 WebKit (Safari): the selector JIT

2.2.1 Hand-written fast paths for single .class/tag/[attr=x]/#id; ID fast path for any selector containing an ID

WebKit's `SelectorDataList` (`Source/WebCore/dom/SelectorQuery.cpp`) classifies the query once at construction and dispatches to a specialized loop:[33]

```
if (m_selectors.size() == 1) {
  const CSSSelector& selector = m_selectors.first().selector;
  if (!selector.precedingInComplexSelector()) {
    switch (selector.match()) {
      case CSSSelector::Match::Tag:   m_matchType = TagNameMatch;
break;
      case CSSSelector::Match::Class: m_matchType = ClassNameMatch;
break;
      case CSSSelector::Match::Exact:
        if (canBeUsedForIdFastPath(selector))
          m_matchType = RightMostWithIdMatch; // [id="name"] pattern
goes here.
```

Single `.class` / `tag` / `[attr=x]` / `#id` selectors get hand-written tight loops over `descendantsOfType<Element>` with no general selector machinery at all. Any selector containing an ID *anywhere* uses `executeFastPathForIdSelector` (rightmost id) or `filterRootById` (an id in an ancestor compound narrows the search root), backed by `TreeScopeOrderedMap` — the same hash-map design as Blink's, given shared ancestry.[33] [34] The `[id="foo"]` case was folded into the ID path in 2016 (r203439, bug 159960) because YUI hammered `querySelector("[id=...]")`, citing Chromium issue 627242.[35] [36]

2.2.2 SelectorCompiler JIT-compiles selectors to machine code; 512-entry cache; whole-selector fallback on unsupported pseudo

Everything escaping the hand-written paths is **JIT-compiled to machine code** under `ENABLE(CSS_SELECTOR_JIT)` — `SelectorCompiler::compileSelector` emits a binary blob per selector; the per-element match is a direct call into generated code:[33]

```
for (Ref element :
descendantsOfTyp
e<Element>(const_cast<ContainerNode&>(searchRootNode))) {
  selectorData.compiledSelector.wasUsed();
  if (selectorChecker(element.ptr())) {   // <-- call into generated
machine code
    appendOutputForElement(output, element);
```

Per WebKit's official write-up (Benjamin Poulain, 2014): "A JIT compiler takes the selector, does all the complicated computations when compiling, and generates a tiny binary blob corresponding to the input selector… When it is time to find if an element that matches the selector, WebKit can just invoke the compiled selector."[37] The compiler is deliberately simple — built on JavaScriptCore's macro-assembler, compilation costs "within one order of magnitude of a single execution of SelectorChecker," amortized over dozens of selectors and hundreds of elements; the measured result was ~2× on common selectors (1100 ms → under 500 ms).[37] Compiled results are cached inside the `SelectorQuery` (`CompilableSingle → CompiledSingle`), and WebKit's `SelectorQueryCache` is a global singleton holding up to **512** entries keyed by `(selector string, parser context, security origin)` with random eviction.[33]

The catch is all-or-nothing: "If any part of a selector is not handle by the CSS JIT, the whole selector is dropped to slow path" (bugs.webkit.org #140818).[38] One unsupported pseudo-class demotes the entire selector to the interpreted `SelectorChecker`.

2.3 Gecko (Firefox): IdentifierMapEntry + right-to-left rationale (~70% rejected on rightmost compound)

Gecko's ID index follows the same architecture: `mozilla::dom::Document`/ShadowRoot keeps `mIdentifierMap`, a `PLDHashTable` of `IdentifierMapEntry` keyed by atom/string hash. `GetIdElement()` is literally `mIdContentList->SafeElementAt(0)` — a hash probe plus an array read — with a `TreeOrderedArray<Element*>` for duplicate IDs and change callbacks (`dom/base/IdentifierMapEntry.h`).[39]

Gecko also supplies the canonical articulation of why every engine matches right-to-left. Boris Zbarsky: matching from the right "gives an obvious starting point and lets you get rid of most of the candidate selectors very quickly" — for Gmail, ~70% of (rule, element) pairs were rejected after examining only the rightmost compound's tag/class/id, and Gecko pre-filters rules with a **hashtable lookup on the element's ID** before attempting any match.[40] You start from one concrete candidate and walk ancestors/siblings, instead of expanding a combinatorial frontier from the left. Modern Gecko (Stylo) buckets style rules by id/class/tag and shares its selector-matching core with `querySelector`.[40]

Concern	Blink	WebKit	Gecko
ID lookup structure	TreeOrderedMap (HeapHashMap, AtomicString key)[28]	TreeScopeOrderedMap (AtomString key)[34]	mIdentifierMap PLDHashTable of IdentifierMapEntry[39]

Concern	Blink	WebKit	Gecko
Class/tag live collections	Cached per (type, name) in NodeListsNodeData; registry bitmask gates invalidation[29] [30]	Live HTMLCollection (bug 147980); bindings inline-cache length[41] [42]	Shared live-list machinery
Parsed selector cache	256 entries per Document, FIFO[31]	512 entries global, random eviction[33]	n/a (style-rule bucketing dominates)
Matcher strategy	Compound state machine + TinyBloomFilter subtree skip + ID-rooted search (2026, CL 7900254)[31] [32]	JIT-compiled machine code per selector; whole-selector slow-path fallback[33] [38]	Right-to-left ancestor walk; id-hashtable pre-filter; ~70% die on rightmost compound[40]
[id=x] attribute selector	ID fast path in compound machine[31]	Folded into ID fast path, r203439[35] [36]	id-bucket pre-filter[40]

2.4 V8 / WebIDL binding layer: the cost you can never inline away

2.4.1 Anatomy of a JS→DOM call: binding callback, argument atomization, C++ work, wrapper + NodeList allocation

Every `document.querySelectorAll(".x")` from JavaScript pays four fixed costs in sequence, before and after any traversal:

1. **Binding dispatch.** Blink's WebIDL compiler auto-generates the glue from IDL files. Per "How Blink works": "When JavaScript calls `node.firstChild`, V8 calls `V8Node::firstChildAttributeGetterCallback()` in v8_node.h, then it calls `Node::firstChild()`."[43] [44]
2. **Argument conversion.** The JS string is converted and **atomized** to an `AtomicString` — interned, hash cached — before any cache can be probed. Cheap per call, never free; string keys dominate the constant factors.[43]
3. **C++ traversal/matching** — everything in §2.1–§2.3.
4. **Result wrapping.** Each returned C++ `Node` needs a V8 wrapper JSObject ("one V8 wrapper per world").[43] `querySelectorAll` additionally allocates the `StaticElementList` snapshot (`Vector` adopted via `StaticElementList::Adopt`) plus its wrapper. Wrappers are cached per node, but a query returning N fresh elements can allocate N wrappers — GC pressure your JS never sees directly.[43]

2.4.2 Why cached JS references win: hidden-class inline cache = 1–2 instructions vs full boundary crossing

On the JS side, V8 optimizes property access via hidden classes (Maps) plus inline caches: the optimizing compiler "can directly inline property accesses if it can ensure a compatible object's structure through the HiddenClass"; the IC machine code is effectively "compare hidden class

pointer… access the property at a hard-coded offset."[45] [46] A field load on your own cached object is ~1–2 machine instructions. A DOM method call can never be fully inlined across the C++ boundary — the dispatch, conversions, and allocations remain on every invocation.

Net effect: a `querySelectorAll` loop is dominated by steps (1), (2), (4) plus the C++ traversal; the JavaScript around it is nearly irrelevant. Caching an element reference in JS eliminates all four steps on subsequent uses. WebKit's engineers reached the same conclusion in their Speedometer post-mortem: "We removed redundant layers of abstraction and made more properties and member functions on DOM objects inline cacheable… We also optimized node lists returned by getElementsByTagName as well as inline caching the length property."[41]

2.5 The machine-level mental model

2.5.1 Hash probe ≈ 1–2 cache misses; tree walk = pointer-chasing, cache-miss-per-hop; bloom filters kill subtrees before walking

Hash lookup (getElementById, any engine). One pointer-chase into a hash bucket; the key's hash is precomputed in the interned string; comparison is pointer equality on a hit. Roughly 1–2 cache misses, no branch count that scales with DOM size. Effectively O(1) with a tiny constant.[28] [34] [39]

JIT-compiled selector match (WebKit). For `div.foo` the generated code is conceptually:

```
load   r1, [element + localName_offset]
cmp    r1, divAtom
jne    fail
load   r2, [element + classList_offset]
call   classList_contains(fooAtom)   ; often itself inlined compares
test   ...
jne    fail
```

A load, an immediate compare, a well-predicted branch — no interpreter dispatch, no virtual calls. That is the entire per-element cost once compiled and cached.[37]

Tree walk (tag/class/qSA without id). Pointer chasing through `firstChild/nextSibling` links. DOM nodes are scattered across the heap by allocation history, so each hop risks an L2/L3 miss (~tens to ~100 cycles), times N nodes. Hence the universal investment in *skip* mechanisms: Blink's per-compound `TinyBloomFilter` rejects a subtree with one AND+compare before descending;[31] WebKit/Gecko pre-filter candidates by id/class buckets so most (element, selector) pairs die on the rightmost compound (~70% on real pages).[40]

Branch prediction. Compound matching is a chain of dependent compares (id? tag? class? attr?). Engines order checks to fail fast on the most selective fact — Blink checks `id_needed` first, then tag, then class, then attribute[31] — because a mispredicted branch (~15–20 cycles) costs as much as several compares. Pseudo-classes break the pattern: they force the general `SelectorChecker` or a whole-selector JIT bailout,[38] and `:nth-child` adds sibling-index bookkeeping (mitigated in Blink by the per-query `NthIndexCache`).[31]

Why "simple" selectors win: they stay on the paths above — pointer compares and bloom bits. Every layer of the four-cost model is either eliminated (parse: string-keyed caches; traversal: hash maps and filters) or reduced to a handful of predicted branches (matching: JIT code, fail-fast compound order). The binding crossing alone is irreducible.

2.5.2 Notable commits/bugs timeline (SelectorQueryCache, WebKit JIT 2014, Blink 2026 rewrite, unified querySelector cache 2026)

Date	Engine	Change	Evidence
2013	Blink	Fork inherits WebKit's SelectorQuery/SelectorQueryCache architecture	selector_query.cc Apple copyright header [31]
2014-03	WebKit	**CSS Selector JIT** lands; ~2× on common selectors; qS/qSA benefit	webkit.org blog 3271 [37]
2015	WebKit	Speedometer work: bindings de-abstracted; NodeList + length inline-cached; "CSS JIT… made querySelector and querySelectorAll much faster"	webkit.org blog 3395 [41]
2015-02	WebKit	CSS JIT :lang() support; "the whole selector is dropped to slow path" on unsupported parts	bugs.webkit.org 140818 [38]
2015-08	WebKit	getElementsByClassName returns live HTMLCollection (spec alignment)	bugs.webkit.org 147980 [42]
2016-07-19	WebKit	r203439: [id=x] reuses the getElementById fast path; motivated by YUI (crbug 627242)	trac r203439, bug 159960 [35] [36]
2026-06	Blink	**Selector query fast-path rewrite** (Steinar H. Gunderson): compound state machine, bloom-	Chromium changelog 2026-06-04; chromium-review 7900254 [32]

Date	Engine	Change	Evidence
		filter subtree skipping, ID-rooted search, matches()/closest() fast paths; reverted once for uninitialized needs_synchronize_attribute, reapplied as CL 7900254 (FillMissingData split)	
2026 (ongoing)	Both	Unified parsed-selector caches at steady state: SelectorQueryCache 256 entries/Document (Blink, FIFO); 512 entries global (WebKit, random eviction)	[31] [33]

Testable predictions for Chapter 3. The architecture above commits Chapter 3's benchmarks to specific outcomes:

- **Map-backed APIs are flat vs DOM size.** `getElementById`, and any `querySelector` containing an `#id`, should show near-constant latency from 10^2 to 10^5 nodes in all three engines.[28] [34] [39]
- **Scan APIs degrade linearly.** Id-less `querySelectorAll` should scale ~linearly with descendant count, slope set by cache-miss-per-hop cost; scoping to a subtree cuts the constant proportionally.[31] [40]
- **Warm beats cold by cache design.** A second identical `querySelector("...")` should be measurably cheaper than the first; dynamically generated selector strings should thrash the 256/512-entry caches and re-pay parse cost.[31] [33]
- **The 2026 Blink rewrite narrows the gap.** Blink's `querySelector('#id')` should converge toward `getElementById` (same TreeOrderedMap, one extra compound check); bloom-filter skipping should make id-less queries sub-linear on sparse-match DOMs — measurable as fewer nodes visited per query, not just lower wall time.[31] [32]
- **Cached JS references dominate everything.** A cached element reference should sit at the ~1–2-instruction floor regardless of engine or DOM size, beating every re-query path by at least the binding-crossing cost.[45] [46]

3. Measured Results: Chrome 150 Sandbox

This chapter reports the original measurements of the investigation. Every number below was measured in this investigation's sandbox (Chrome 150.0.7871.181, Blink+V8, headless; median of 10 samples, two full suite runs × 5 repetitions of ≥300 ms each after 200 ms warmup; case order rotated per repetition; a global XOR sink guarded against dead-code elimination; `gc()` ran between suites; timed loops were read-only, no layout thrash). Fixtures were synthetic documents of 261 (small), 2,061 (medium), and 20,097 (large) actual nodes including injected targets, ~8 levels deep, div/span/input mix, classes `.a/.b/.c` round-robin, ids on ~1% of nodes, `data-x="1"` (plus class `.dx`) on 10%; a separate 500-deep div chain served the traversal cases. Tables report median ops/sec with CV (stdev/median across the 10 samples) and a rel column normalized to the fastest variant in the same table (1.00 = fastest). Two independent full runs agreed within ±10% on nearly all medians (worst ~13%). A handful of cases show elevated CV (10–18%) from occasional GC pauses inside 300 ms samples; medians were stable across runs. Absolute numbers are machine-specific; ratios are the signal.

Chapter 2 predicted three patterns: map-backed lookups flat in document size, tree-scan APIs degrading ~linearly, and a narrowed getElementById-vs-querySelector gap after the 2026 Blink SelectorQuery rewrite. Each section below confirms or refutes those predictions against the data. (Chapter 4 contrasts these numbers with historically published figures; no history is discussed here.)

3.1 Single-hit lookups: ID, class, tag

3.1.1 getElementById vs querySelector('#id') vs querySelectorAll('#id'): 1.18×/2.3× spread, O(1) flat across 261→20,097 nodes

variant	small ops/s (CV)	medium ops/s (CV)	large ops/s (CV)	rel small	rel medium	rel large
getElementById	5.80M (2%)	5.85M (4%)	6.02M (2%)	1.00	1.00	1.00
querySelector(#target)	4.81M (1%)	5.00M (2%)	5.09M (2%)	0.83	0.86	0.85
querySelectorAll(#target)	2.59M (4%)	2.64M (5%)	2.64M (8%)	0.45	0.45	0.44

All three variants are flat within noise from 261 to 20,097 nodes — getElementById even drifts upward (5.80M → 6.02M ops/s, 1.04×), ruling out any residual tree walk. Its margin over `querySelector('#id')` is only 1.18× at the large fixture, far narrower than the multi-x gap library folklore assumes; both resolve through the same per-document id map, and the residual difference is selector-parsing and entry-point overhead, not lookup cost (see Chapter 2 for the map mechanics). This confirms the Chapter 2 prediction of a narrowed id gap post-Blink-rewrite. `querySelectorAll('#id')` is 2.3× slower at every size — a constant factor from static NodeList allocation — yet still O(1), so even the "bad" id path never degrades with document growth. CVs of 1–8% make the spread solidly significant.

3.1.2 getElementsByClassName (O(1), 9.3M ops/s @20K) vs querySelectorAll('.c-target') (10× degradation, 57× slower @20K)

variant	small ops/s (CV)	medium ops/s (CV)	large ops/s (CV)	rel small	rel medium	rel large
getElementsByClassName(.length)	9.39M (2%)	9.39M (17%)	9.28M (5%)	1.00	1.00	1.00
getElementsByClassName[0]	4.80M (2%)	4.91M (18%)	4.83M (4%)	0.51	0.52	0.52
querySelector(.c-target)	3.98M (2%)	3.66M (14%)	3.52M (3%)	0.42	0.39	0.38
querySelectorAll(.c-target)	1.61M (2%)	1.27M (4%)	163.9K (7%)	0.17	0.14	0.02

The table splits cleanly into cache-backed and scan-backed rows. getElementsByClassName holds at 9.3M ops/s at 20K nodes (0.99× vs small) — O(1) via Blink's tree-indexed class cache. Reading `[0]` instead of `.length` halves throughput (0.51–0.52 rel): indexed access forces element materialization rather than a cheap count, but stays flat. Single-hit `querySelector('.c-target')` is ~1.4× slower than `gEBCN[0]` and also roughly O(1) here (0.88× large/small), because Blink's document-level class cache locates a rare class without a full scan. The outlier is `querySelectorAll('.c-target')`: 1.61M → 163.9K ops/s, a 10× degradation small→large, ending 57× slower than getElementsByClassName at 20K nodes — building a static list forces a full tree walk. Elevated CVs (14–18%) at medium are GC noise; medians were stable across runs.

3.1.3 getElementsByTagName vs querySelectorAll('span'): 38× / 273× / 3292× by size

variant	small ops/s (CV)	medium ops/s (CV)	large ops/s (CV)	rel small	rel medium	rel large
getElementsByTagName(span)	11.09M (7%)	11.01M (9%)	11.46M (4%)	1.00	1.00	1.00
querySelectorAll(span)	290.5K (2%)	40.3K (4%)	3.5K (6%)	0.03	0.00	0.00

This is the widest gap measured in the investigation. getElementsByTagName('span') delivers ~11M ops/s at every fixture size (large/small 1.03×) — perfectly O(1), served from the tag-name index. `querySelectorAll('span')` pays a full tree scan plus static-list construction, so throughput collapses with node count: 290.5K → 40.3K → 3.5K ops/s. The margins are 38× at small, 273× at medium, and 3292× at large (computed from unrounded medians in results.json; the table's 3.5K is rounded). No selector-level optimization can close a gap of this shape: qSA

must visit every node while gEBTN visits none. This is the strongest confirmation of the Chapter 2 scan-vs-map dichotomy, and why tag-name hot paths should never route through qSA regardless of expected document size.

3.2 Collection iteration and the caching multiplier

3.2.1 Iteration: cached-length live loop fastest; 1.23× over uncached .length; 1.37× over static NodeList; 2.4× over NodeList.forEach

variant	medium ops/s (CV)	large ops/s (CV)	rel medium	rel large
live HTMLCollection (cached length)	11.7K (2%)	1.1K (2%)	1.00	1.00
live HTMLCollection (c.length each iter)	9.4K (5%)	873 (6%)	0.80	0.82
static NodeList (querySelectorAll)	8.3K (1%)	780 (3%)	0.71	0.73
static NodeList forEach	5.0K (3%)	438 (6%)	0.43	0.41

All four variants scale ~linearly downward with fixture size because iteration itself is O(n); the differentiator is constant factor. A classic `for` loop over a live HTMLCollection with the length hoisted into a local is fastest at both sizes. Re-reading `.length` per iteration costs 1.23×: on a live collection each access re-validates the cached snapshot, a real (if small) C++ round-trip per element. Iterating the static NodeList from querySelectorAll — qSA construction included in the timed region — is 1.37× slower, quantifying the list-building tax amortized over a single pass. `NodeList.forEach` trails at 2.4× behind the leader, callback invocation overhead dominating at ~2K–20K elements. CVs of 1–6% are among the tightest in the suite, so all three ratios are reliable. Rule: cache the collection *and* its length; avoid forEach on hot paths.

3.2.2 ×100 re-query: cached live collection = 6.1× over fresh gEBCN, ~13,700× over fresh qSA at large size

variant	medium ops/s (CV)	large ops/s (CV)	rel medium	rel large
cache live collection x100 reads	415.6K (7%)	420.6K (1%)	1.00	1.00
re-query gEBCN x100	68.2K (4%)	68.9K (1%)	0.16	0.16
re-query qSA x100	431 (14%)	31 (3%)	0.00	0.00

The benchmark performs 100 accesses per timed unit, either by re-querying or by reading a collection cached once. Caching wins even though the HTMLCollection is "live": holding the reference costs 6.1× less than re-calling getElementsByClassName 100 times, proving that construction — not liveness maintenance — is the expense; liveness only charges when the DOM mutates, and these loops are read-only. Against re-running querySelectorAll 100 times the multiplier reaches 13,715× at the large fixture (420.6K vs 31 ops/s), each qSA re-executing a full O(n) scan. Note the flat medium→large profile of both gEBCN rows (68.2K → 68.9K, 415.6K → 420.6K) versus qSA's 431 → 31 collapse — the §3.1 map-vs-scan law amplified 100-fold. The 14% CV on medium qSA is GC noise on a near-floor measurement and does not affect the ordering.

3.3 Tree traversal: closest(), children, siblings

3.3.1 closest() 4.1× over manual parentNode+classList loop, 3.7× over matches() loop; selector complexity irrelevant (0.99×)

variant	chain-500 ops/s (CV)	rel
closest(.ancestor-at-depth-50)	793.3K (2%)	1.00
closest(.ancestor[data-x="1"])	788.1K (2%)	0.99
closest(div.ancestor)	492.1K (2%)	0.62
manual loop + matches()	211.7K (2%)	0.27
manual loop + classList.contains	193.2K (2%)	0.24

On a 500-deep div chain with the match 50 levels above the leaf, native `closest()` beats the hand-rolled `parentNode` loop by 4.1× when the manual loop guards with `classList.contains` and by 3.7× with `matches()`. The engine walks the same 50 ancestors in C++ with no JS↔C++ boundary crossing per step, while the manual versions pay a property access and a call per level (see Chapter 2 for the boundary-cost model). Selector complexity is nearly irrelevant: `closest('.ancestor[data-x="1"]')` runs at 0.99× of the plain class selector — the walk dominates, not the test. The one complexity effect is inverted: `closest('div.ancestor')` drops to 0.62× because the compound fails fast at intermediate divs lacking the class, changing early-out behavior. All CVs are 2%; the 4× ordering is unambiguous. Libraries still shipping manual ancestor loops should delete them.

3.3.2 firstElementChild 2.1× over children[0], 3.7× over childNodes[0]; nextElementSibling 2.3× over nextSibling skip, ~2.2×/2.4× over TreeWalker/NodeIterator

variant	unit ops/s (CV)	rel
el.firstElementChild	15.74M (2%)	1.00
manual firstElementChild polyfill	10.66M (2%)	0.68
el.children[0]	7.33M (2%)	0.47

variant	unit ops/s (CV)	rel
el.childNodes[0] (text node!)	4.26M (3%)	0.27

variant	unit ops/s (CV)	rel
nextElementSibling chain	252.5K (1%)	1.00
TreeWalker(SHOW_ELEMENT)	116.5K (4%)	0.46
nextSibling chain (skip non-elements)	108.1K (3%)	0.43
NodeIterator(SHOW_ELEMENT)	105.7K (3%)	0.42

Two unit fixtures back these ratios. For first-child access (text nodes deliberately present before the first element), `firstElementChild` at 15.74M ops/s beats `children[0]` by 2.1× — the latter constructs a full HTMLCollection to read one entry — and beats `childNodes[0]` by 3.7×; note `childNodes[0]` here returns the *text* node and is not semantically equivalent, included only to price the trap. Even a manual firstChild/nextSibling polyfill (10.66M ops/s) beats `children[0]`. For walking 200 element siblings with interleaved text nodes, the `nextElementSibling` chain wins at 252.5K walks/s: 2.34× over a `nextSibling` chain with JS-side nodeType filtering, and ~2.2× over TreeWalker (116.5K) and 2.4× over NodeIterator (105.7K), whose iterator allocation dominates at this range. Element-only accessors keep filtering in C++; every JS-visible alternative pays per-step boundary or allocation costs.

3.3.3 Delegation guard: tagName === > classList.contains (0.63×) > matches() (0.44×)

variant	unit ops/s (CV)	rel
el.tagName === "DIV"	23.97M (3%)	1.00
el.classList.contains("cls")	15.17M (3%)	0.63
el.matches(".cls")	10.46M (3%)	0.44

Delegation guards run once per bubbled event, so their relative cost matters at high event rates. A plain `tagName ===` comparison is fastest at 23.97M ops/s: a cached-string identity check with no selector machinery. `classList.contains` runs at 0.63× of that, `matches('.cls')` at 0.44× — i.e. `matches()` is ~1.4× slower than `classList.contains` for a pure class guard, the price of parsing and evaluating a selector where a token-set lookup suffices. The best-to-worst spread is 2.3×, yet the honest caveat is that all three exceed 10M ops/s: guard choice matters only in genuinely hot dispatch loops, never as a correctness decision. CVs are a uniform 3%; the ordering tagName > classList.contains > matches() is stable and matches Chapter 2's cost model (string compare < token lookup < selector match).

3.4 Exotic paths: XPath, attribute selectors, named access

3.4.1 XPath slowest: 26× over qSA, ~70,000× over class lookup @20K

variant	medium ops/s (CV)	large ops/s (CV)	rel medium	rel large
getElementsByClassName(dx) [equiv set]	11.47M (2%)	11.31M (3%)	1.00	1.00
querySelectorAll([data-x="1"])	74.7K (2%)	4.2K (6%)	0.01	0.00
document.evaluate(//*[@data-x="1"])	1.6K (3%)	162 (3%)	0.00	0.00

`document.evaluate('//*[@data-x="1"]')` is the slowest lookup measured in this investigation: 1.6K ops/s at medium, 162 at large — 26× slower than the equivalent `querySelectorAll('[data-x="1"]')` at large and 69,702× slower than getElementsByClassName on an equivalent class at 20K nodes. XPath in Blink combines an interpreted expression engine, its own result-set snapshot object, and zero index acceleration: every evaluation is a full O(n) walk with heavyweight per-node dispatch (see Chapter 2). Mirroring the attribute set with a class buys four to five orders of magnitude (11.31M vs 162 ops/s at large) — the single largest lever in this chapter when the markup is under the consumer's control. The XPath row degrades ~10× and the qSA('[data-x]') row ~18× medium→large, both consistent with the linear-scan law of §3.5.2.

3.4.2 qSA('[data-x="1"]') 19×/12× over gEBTN('')+manual filter; getElementById 2.8× over window['id'], 8.8× over document.forms['name']*

variant	medium ops/s (CV)	large ops/s (CV)	rel medium	rel large
querySelectorAll([data-x="1"])	75.5K (3%)	4.6K (6%)	1.00	1.00
getElementsByTagName(*) + manual filter	3.9K (3%)	382 (2%)	0.05	0.08

variant	unit ops/s (CV)	rel
getElementById("namedId")	8.05M (2%)	1.00
window["namedId"]	2.89M (1%)	0.36
document.forms["benchForm"]	912.2K (1%)	0.11

The attribute-selector table refutes the old advice to "grab everything and filter in JS": `querySelectorAll('[data-x="1"]')` beats `getElementsByTagName('*')` plus a manual `getAttribute` filter by 19× at medium and 12× at large. Both are O(n) scans, but the manual variant materializes a live collection of every element and crosses the JS↔C++ boundary for each `getAttribute` call, while qSA evaluates the predicate inside the engine. The named-access table prices legacy shortcuts: getElementById (8.05M ops/s) beats `window['namedId']` named-property resolution by 2.8× and `document.forms['benchForm']` by 8.8× — the named paths route through the named getter on the Window/HTMLFormElement objects rather than the id map, and `document.forms` additionally constructs the forms collection. Both legacy paths are also spec-fragile; there is no performance excuse for them.

3.5 Selector complexity ladder and scaling behavior

3.5.1 Ladder @20K: span 3.6K > .c 3.2K > [data-x] 3.1K > span.c 1.9K > div > span.c 1.8K > div span.c 1.5K > :nth-child(3) 1.0K ops/s

variant	large ops/s (CV)	rel
span	3.6K (3%)	1.00
.c	3.2K (9%)	0.88
[data-x]	3.1K (16%)	0.85
span.c	1.9K (18%)	0.52
div > span.c	1.8K (16%)	0.50
div span.c	1.5K (17%)	0.42
:nth-child(3)	1.0K (6%)	0.29

All seven selectors are full tree scans via querySelectorAll on the 20K-node fixture, so the ladder prices constant factors alone — the simplest-to-hardest spread is only 3.5×, nothing like the orders-of-magnitude gaps in §3.1. Simple selectors cluster tightly (3.1–3.6K ops/s); compounding (`span.c`) nearly halves throughput because each candidate element runs two tests; combinators cost a further 4–19% (`div > span.c` at 0.50, `div span.c` at 0.42 — the descendant variant's ancestor walk edges the child variant's single parent check in this 8-level tree); `:nth-child(3)` is worst at 0.29 rel, the structural pseudo-class forcing sibling-position evaluation per candidate. CVs of 16–18% on the compound rows are the suite's GC-noise caveat; medians held across both runs. Lesson: simplify selectors for constant-factor wins, but never expect selector tuning to fix an algorithmically wrong API choice.

3.5.2 Scaling law: map-backed APIs flat (0.99–1.06×); tree-scan APIs degrade ~linearly (qSA span 83×)

variant	small	medium	large	med/small	large/small
getElementById	5.80M	5.85M	6.02M	1.01x	1.04x
querySelector (#target)	4.81M	5.00M	5.09M	1.04x	1.06x

variant	small	medium	large	med/small	large/small
querySelectorAll(#target)	2.59M	2.64M	2.64M	1.02x	1.02x
getElementsByClassName(.length)	9.39M	9.39M	9.28M	1.00x	0.99x
getElementsByClassName[0]	4.80M	4.91M	4.83M	1.02x	1.01x
querySelector(.c-target)	3.98M	3.66M	3.52M	0.92x	0.88x
querySelectorAll(.c-target)	1.61M	1.27M	163.9K	0.79x	0.10x
getElementsByTagName(span)	11.09M	11.01M	11.46M	0.99x	1.03x
querySelectorAll(span)	290.5K	40.3K	3.5K	0.14x	0.01x

This is the chapter's central table: it sorts every lookup API by asymptotic behavior across 77× fixture growth (261 → 20,097 nodes). The map-backed group — getElementById, both getElementsByClassName variants, getElementsByTagName, and both id-selector querySelector forms — holds within 0.99–1.06×, flat inside noise: O(1) lookups served by Blink's per-document indexes and caches, confirming the Chapter 2 scaling prediction. The tree-scan group degrades with size: `querySelectorAll('span')` falls 83× small→large (0.01×, slightly exceeding the 77× node growth due to larger static-list allocation), `querySelectorAll('.c-target')` falls 10×, XPath (§3.4.1) ~10× medium→large — all ~linear. The only anomaly is `querySelector('.c-target')` at 0.88×: nominally cache-assisted, it drifts slightly, consistent with lower cache-hit probability as the tree grows. Corollary: any API whose throughput trends down with n is borrowing against document growth.

3.6 Counter-intuitive results: scoped querySelector slower than document-level (1.28×) — fixture-dependent class-cache effect

variant	medium ops/s (CV)	large ops/s (CV)	rel medium	rel large
document.querySelector(.deep-target)	3.56M (13%)	3.45M (17%)	1.00	1.00
scopeRoot.querySelector(.deep-target)	2.66M (18%)	2.69M (7%)	0.75	0.78

Conventional wisdom says "scope your queries to a subtree"; this investigation measured the opposite. `document.querySelector('.deep-target')` beat

`scopeRoot.querySelector` by 1.28× at the large fixture (3.45M vs 2.69M ops/s). The mechanism is the document-level class cache: for a rare class, Blink resolves the document-wide query through the cache without walking the tree, whereas the scoped call must walk the subtree because element-level querySelector cannot use the document cache (see Chapter 2). Two caveats from the RESULTS data must be reproduced. First, the result is fixture-dependent: the scoped root sat near the top of the document and the target class was rare, so the cache hit decisively; with a common class or a deeper, smaller scope the outcome can differ — scoping only pays when document-level caches miss and the subtree is small. Second, CVs here are among the suite's highest (7–18%, GC pauses inside 300 ms samples), but both rows are flat medium→large and the 1.28× ordering held in both full runs. Treat scoping as a cache-miss optimization, not a universal one.

4. Historical & Cross-Browser Evidence

Published DOM-query benchmarks span fourteen years of engine history, and their ratios disagree with each other far more than they disagree with folklore. That disagreement is not noise to be averaged away; it is the chapter's subject. Every number below carries its browser, version, and year, because a ratio without a UA string is not evidence — it is an anecdote with arithmetic.

4.1 Published ratios over time

Comparison	Ratio	Browser / Year	Source
gEBI vs qS('#id'), fixed loop	1.08×	Chrome, 2019	[47]
gEBI vs qS	1.7× / 2.7×	Chrome 73 / 75, 2019	[48] [49]
gEBI vs qS, 100k-node DOM	8.9× FF / 4.7× Chromium	FF 92 / Chromium 93, 2021	[50]
gEBI vs qS	1.9×	Chrome 93, 2021	[51]
gEBI vs qS	1.4×	Chrome 115, 2023	[52]
gEBI vs qS	1.8× (mobile)	FF Mobile 129, 2024	[53]
gEBI vs qS('#id')	6.4× / 4.3× / 2.1×	Chrome 145/146/126, 2026 runs	[54] [55] [56]
gEBCN vs qSA('.class')	~29×	Chrome, ~2016 (jsperf)	[4]
gEBCN vs qSA('.class')	~51×	Chrome 68, 2018	[57]
gEBCN vs qSA('.class')	~4.5×	Chrome 100, 2022	[58]
qSA vs TreeWalker (filter H1)	qSA ~3.9× faster	Chrome; Safari 26 ~2.3×	[59]
TreeWalker vs qSA('*')	TreeWalker ~2.3× faster	Chrome, ~2024–25	[60]
NodeIterator JS-filter	32× / 13× / ~20×	Chrome / FF / Safari	[61]

Comparison	Ratio	Browser / Year	Source
vs TreeWalker param-filter			
closest() vs manual while-loop	parity (~1.04×)	Chrome 131, 2024; Chrome 109, 2023	[62] [63]

4.1.1 getElementById vs qS('#id'): folklore 10–100× vs measured 1.1×–9×

The folklore ratio — "`getElementById` is ten to a hundred times faster than `querySelector`" — circulates in blog posts and interview prep sheets without provenance. The published record does not support it. As the table shows, community runs on measurethat.net and its mirrors cluster between 1.4× and 6.4× across eight Chrome generations (2019–2026)[55] [54] [56] [49] [48] [51] [52]; a 2019 fixed-loop timing put the id case at just 1.08× (691 ms vs 749 ms)[47]. The earliest archived comparison, from the jsperf era in 2012, reported only an ordinal ranking, with anomalies such as Opera 12.11 beating everyone at qSA and Safari 6 being "immensely fast" at getElementById[64].

The most rigorous published study is Wesley Aptekar-Cassels' 2021 experiment on a 100k-element DOM with a select-all loop: Firefox 92 returned 7 ms for getElementById against 62 ms for querySelector (**8.9×**); Chromium 93 on the same machine returned 44 ms vs 206 ms (**4.7×**), with markedly higher variance on the querySelector side[50]. What drives this number to the top of the range: a huge DOM plus repeated identical selectors. The honest summary of the corpus is **1.1×–9×** — the "10–100×" claim appears only in buggy tests (§4.2.1) or in extrapolation from large-DOM setups like Aptekar-Cassels', which tops out at 8.9×.

4.1.2 Class lookup gap collapse: 51× → 4.5×

The class-selector comparison is the clearest record of an engine closing a real gap. In the jsperf era (~2016, Chrome), getElementsByClassName posted 1,138,018 ops/s against querySelectorAll's 39,033 — **29×**[4]. **A 2018 run on Chrome 68 widened it to 51×** (3,567,748 vs 69,880 ops/s)[57]. Then the same suite, re-run on Chrome 100 in 2022, collapsed to **~4.5×** (4,205,284 vs 944,033 ops/s)[58]; a 2019 fixed-iteration timing found only **1.9×** even then[47]. The mechanism is documented engine-side: Chrome's 2024 Speedometer 3 work streamlined common querySelector selector paths[65]. A 51× figure from Chrome 68 is not reproducible on Chrome 146+; quoting it today without the UA string is a category error.

4.1.3 Contradictory TreeWalker results; the NodeIterator callback cliff

Traversal is where the published corpus contradicts itself outright, and both sides deserve to be shown. Filtering H1 elements, querySelectorAll beat TreeWalker **~3.9×** (735,159 vs 196,298 ops/s, Chrome; Safari 26 confirms ~2.3×)[59]. Collecting all elements, TreeWalker beat querySelectorAll('*') **~2.3×** (5,856,998 vs 2,580,535 ops/s, Chrome)[60]. The contradiction resolves on inspection: the first fixture filters a sparse match set, the second walks everything; DOM size and setup differ, and neither suite controls for the other's regime. Treat "TreeWalker is faster/slower than qSA" as fixture-dependent, not a property of the APIs.

One traversal result is unambiguous across all three engines: NodeIterator with a JavaScript filter *function* is a cliff — **32×** slower than TreeWalker with a filter parameter on Chrome

(29,576 vs 946,711 ops/s), 13× on Firefox, ~20× on Safari[61]. The penalty is the per-node JS callback crossing the binding boundary, not the walker itself. Manual `nextSibling` traversal functions fare similarly against TreeWalker's native filter (~5.4×, Chrome 144, 2026)[66].

4.2 Why numbers differ across eras and browsers

4.2.1 Engine evolution, fixture sensitivity, and the '.test'-with-dot bug

Two forces move published ratios. The first is genuine engine change: Blink's 2024 selector streamlining[65] and the 2026 unified querySelector() cache — "a unified querySelector() cache across the rendering engine to cut down on redundant DOM lookups"[67] — each compress repeat-query gaps. The second is fixture sensitivity: Aptekar-Cassels' 8.9× depends on a 100k-node DOM, while measurethat runs on toy fixtures show 1.4–2.7× for the same comparison[56] [49] [48] [52]. And some published numbers are simply wrong. A widely circulated "~10×" gEBCN-vs-qSA console.time test (2023) passed `'.test'` — dot included — to `getElementsByClassName`, which silently returned an empty collection; the "winner" performed zero work[7]. The same thread popularized the claim that querySelector* is O(n) while getElement* is O(1)[7] — an oversimplification given engine ID/class caches, best treated as folklore pending controlled verification.

4.2.2 Microbenchmark pitfalls catalog

Pitfall	Mechanism	Documented magnitude	Source
Dead code elimination	JIT removes result-free loops; DOM calls are partly protected by the JS⇄C++ boundary, but identical-selector hoisting distorts instead	discarded results may vanish entirely	Egorov 2012[68]; mitata[69]
Loop-invariant hoisting / selector caching	repeated querySelector('#same') hoisted or served from Blink's 2026 cache	inflates repeat-query suites	Chromium blog[67]
Warmup / JIT tiering	cold interpreter passes counted; Firebug historically disabled Firefox's JIT	interpreter-speed numbers reported as optimized	Bynens 2010[70]; Benchmark.js[71]
Timer granularity	post-Spectre performance.now() reduction; Firefox rounds to 2 ms from FF59	sub-100 ns calls individually unmeasurable	MDN via SO[72]
Layout thrashing	geometry reads	1.5–2.3× speedup	Stack Insight study[73]

Pitfall	Mechanism	Documented magnitude	Source
contamination	interleaved with style writes force synchronous layout	from batching alone (Chromium 147, 2026)	
Live vs static collections	re-reading .length of a live HTMLCollection forces re-resolution	~4× penalty; 15.3M → 3.85M ops/s (Edge/Chrome 148)	measurethat #28558[74]
Selector-string bugs	'.test' with dot → empty collection; asymmetric query counts	spurious "~10×"	richardmyu[7]; SO #57159219[47]
Setup / GC contamination	DOM construction or allocation inside the timed region; GC between samples	margin-of-error reporting required	Benchmark.js practice[69] [71]

Egorov's standing advice — "In general I advise against microbenchmarking"[68] — applies with a DOM-specific twist: because query calls cross the binding boundary, classic DCE is rare, but hoisting and engine-side caching produce the identical distortion by a different door[67]. And any suite that touches styles or geometry between queries is measuring the renderer, not the selector engine; the Chromium-147 study's protocol (30 trials, discard 5 warmup, accept 25 at CV<15%, median) is a reasonable template[73].

4.3 Reconciling published numbers with our Chrome-150 measurements

Chapter 3's sandbox numbers sit at the extreme low end of every published range, and the gap is explicable rather than alarming. Our getElementById advantage over qS('#id') measured **1.18×**, against 4.3–6.4× in 2026-era community runs[55] [54] and 8.9×/4.7× in the 2021 large-DOM study[50]. Two mechanisms close most of the distance. First, the 2026 Blink SelectorQuery rewrite and unified querySelector() cache explicitly target redundant DOM lookups[67]: community suites that re-query an identical selector string — the dominant pattern in measurethat loops — now hit cache, compressing the repeat-query gap that produced the 4–9× era. Our sandbox alternates selectors and controls for cache warmth, so 1.18× is the *cold-path* residual: the true marginal cost of selector parsing and dispatch once caching is neutralized. Second, the 2024 Speedometer 3 selector streamlining[65] had already removed the pathological slow paths that made 2016–2018 Chrome show 29–51× on classes[4] [57] [58].

The same logic reconciles the class and tag results in the opposite direction. Our gEBCN-over-qSA('.c-target') ratio of **57×** at 20K nodes exceeds every post-2022 published figure (~4.5× on Chrome 100[58]) — because published suites use small fixtures where qSA's per-call overhead dominates, while at 20K nodes qSA's static-list construction cost scales and gEBCN's cached class index does not. The published 4.5× and our 57× are the same curve sampled at different DOM sizes. Likewise gEBTN-over-qSA('span') at **3,292×** is the large-fixture extreme of the trend in older suites[49] [75].

For closest(), the inversion runs the other way. Both public suites show parity — 1.04× on Chrome 131 (2024)[62], parity on Chrome 109 (2023)[63] — but both walk shallow ancestors in tiny DOMs, where the while-loop's per-step JS overhead is negligible. Our **4.1×** closest() advantage was measured at realistic ancestor depths, where the manual loop pays a JS↔C++ boundary crossing per hop and closest() walks natively. Published parity and our 4.1× are both correct — for different depths. The general lesson of this chapter: published ratios are points on curves parameterized by DOM size, depth, cache warmth, and engine vintage. Quote them with their coordinates, or not at all.

5. Decision Matrix: Which API for Which Situation

Chapters 1–4 established the semantics, the engine mechanics, the measured ratios, and the historical drift. This chapter converts that evidence into orders. Every directive below answers three questions in sequence: **what do you know about the target** (identity, structure, or only a predicate), **how do you access the result** (once, repeatedly, or iteratively), and **what is the document regime** (size, mutation rate). Answer them honestly and §5.4 hands you the API.

One global law precedes everything else: any API whose throughput degrades with node count is borrowing against document growth. In this investigation's sandbox (Chrome 150), the map-backed group held flat at 0.99–1.06× across a 77× fixture growth while `querySelectorAll('span')` collapsed 83×. When in doubt, choose the API whose cost curve you cannot make worse by adding markup.

5.1 By target knowledge: you know the ID / class / tag / structure / nothing

*5.1.1 Known ID → getElementById (document or shadow root); known class/tag at scale → getElementsBy**

Known ID: use `getElementById`. Measured in this investigation's sandbox (Chrome 150), it beats `querySelector('#id')` by 1.18× at 20K nodes, both O(1) flat from 261 to 20,097 nodes. The gap is narrow — far narrower than the 4–9× of the 2021–2026 published record[50] [54] — because both resolve through the same per-tree id map; the residual is selector parse and dispatch overhead, not lookup cost. The decisive reasons are therefore not the 1.18× but the ergonomics of the fast path: no parse, no `SyntaxError`, no `CSS.escape()`, and it exists on `ShadowRoot`, since `getElementById` lives on the `DocumentOrShadowRoot` mixin and ids are unique per tree.[1] Inside a shadow root, `shadowRoot.getElementById(...)` is the only indexed option; no document-level call reaches in.[1] Never take the legacy shortcuts: `window['id']` measured 2.8× slower and `document.forms['name']` 8.8× slower, both routing through named getters instead of the id map — and both are spec-fragile besides.

Known class or tag at scale: use `getElementsByClassName` / `getElementsByTagName`. gEBCN held 9.3M ops/s at 20K nodes while `querySelectorAll('.c-target')` fell to 163.9K — 57× slower; gEBTN held ~11M ops/s while `querySelectorAll('span')` collapsed 38×/273×/3292× by size, the widest gap measured. The reason is the scan-vs-map dichotomy: Blink serves these from tree-indexed class/tag caches with invalidation-based liveness,[29] [30] while qSA must walk every candidate and build a static list. The measured 57× exceeds the

post-2022 published ~4.5×[58] only because the published fixtures are small — same curve, different DOM size. A tag-name hot path routed through qSA is a bug at any size.

5.1.2 Known structure → direct property walk (firstElementChild/nextElementSibling) beats any query

If you know where the node is, walk there; do not query. `firstElementChild` measured 2.1× over `children[0]` (which constructs a full HTMLCollection to read one entry) and 3.7× over `childNodes[0]` — and `childNodes[0]` is additionally a semantic trap, returning the text node. A `nextElementSibling` chain beat a `nextSibling`-with-nodeType-skip loop by 2.3× and both TreeWalker and NodeIterator by ~2.2×, whose iterator allocation dominates at this range. Element-only accessors keep the filtering in C++; every JS-visible alternative pays a per-step binding crossing or an allocation. For ancestor search, `closest()` beat the hand-rolled `parentNode` loop by 4.1× (`classList.contains` guard) and 3.7× (`matches()` guard) on a 500-deep chain — the engine walks the same 50 ancestors with no JS↔C++ crossing per step, and selector complexity inside `closest()` was nearly free (0.99× for a compound vs plain class). Delete manual ancestor loops. In delegation guards, the order is `tagName ===` > `classList.contains` (0.63×) > `matches()` (0.44×): cached-string identity, then token-set lookup, then selector parse — though all three exceed 10M ops/s, so this matters only in genuinely hot dispatch.

5.1.3 Complex relational criteria → querySelector with simple key compound; attribute presence → qSA('[attr]') never ''+filter*

When all you have is a predicate, querySelector(All) is the right tool — but write the selector for the matcher. Engines match right-to-left: the rightmost compound filters every candidate, and everything left of the rightmost combinator is per-survivor verification, not candidate reduction.[6] The sandbox selector ladder prices this: at 20K nodes the full spread from `span` (3.6K ops/s) down to `:nth-child(3)` (1.0K) is only 3.5× — constant factors, never the orders of magnitude of §5.1.1. So: put the most selective, indexable term in the key position, avoid structural pseudo-classes (sibling-position bookkeeping per candidate), and expect tuning to win you 2–3× at most. Selector simplification never fixes an algorithmically wrong API choice.

`#a .b` is never faster than `.b`. Both face the identical candidate set; the id prefix only adds an ancestor walk per `.b` survivor.[6] The correct way to shrink candidates is to shrink the root: `getElementById('a').querySelectorAll('.b')` — with the §5.3.2 caveat that scoping is a cache-miss optimization, not a universal one. And prefix `:scope` on element-rooted queries, which match against the whole tree before filtering to descendants.[15]

Attribute presence or equality: use `qSA('[data-x="1"]')`, never `getElementsByTagName('*')` plus a manual filter. The measured margin is 19× at medium and 12× at large. Both are O(n) scans, but the manual variant materializes a live collection of every element and crosses the JS↔C++ boundary for each `getAttribute`, while qSA evaluates the predicate inside the engine. The old "grab everything and filter in JS" advice is refuted, not merely outdated.

Never XPath. `document.evaluate('//*[@data-x="1"]')` measured 26× slower than the equivalent qSA and ~70,000× slower than gEBCN on an equivalent class at 20K nodes — an interpreted expression engine with its own snapshot object and zero index acceleration. If the markup is yours, mirror the attribute with a class: that single change is worth four to five orders of magnitude, the largest lever in this book.

5.2 By access pattern: one-shot vs repeated vs iteration

5.2.1 One-shot single hit → qS over qSA[0]; repeated → cache reference or live collection (6.1×/13,700×)

One shot, one hit: use `querySelector`, never `querySelectorAll(...)[0]`. qS bails at the first match (`kShouldOnlyMatchFirstElement`); qSA walks the whole subtree and allocates the full static list to hand you one entry.[31] The same laziness argument makes `gEBCN('x')[0]` acceptable for a single class hit: creation is ~O(1) registration, cost deferred to access.[7]

Repeated access: cache, and cache the right thing. The ×100 re-query benchmark is unambiguous: holding one cached live collection beat 100 fresh gEBCN calls by 6.1× and 100 fresh qSA calls by ~13,700× at the large fixture. Construction — not liveness maintenance — is the expense; liveness only charges when the DOM mutates, and read-only loops never pay it. Best of all is a cached element reference in JS: a field load on your own object is ~1–2 machine instructions via hidden-class inline caches, while a DOM method call can never be inlined across the binding boundary.[45] [46] Re-querying an identical selector string also leans on engine parse caches (Blink: 256 entries per document; WebKit: 512 global),[31] [33] so dynamically generated selector strings thrash them — validate and hoist selector strings out of hot paths.[5]

5.2.2 Iterate-then-mutate → snapshot live collection first; read-only hot loops → cached-length live loop

Iterating while mutating: snapshot first. Removing `live[i]` shifts every later element down one index and silently skips successors; growing while testing `i < live.length` never terminates.[7] Take a static snapshot — `Array.from(live)` or a qSA static NodeList — then mutate freely. Iterating a mutating tree with a cursor: `NodeIterator`, whose reference node is adjusted by pre-remove steps, not `TreeWalker`, whose cursor strands on a detached node.[18] [8]

Read-only hot iteration: classic `for` loop over the live collection, length hoisted. Measured: cached-length live loop 1.00; re-reading `.length` per iteration 1.23× slower (each access re-validates the cached snapshot — a C++ round trip per element); iterating a freshly built static NodeList 1.37× slower (list construction amortized over one pass); `NodeList.forEach` 2.4× slower (callback overhead at ~2K–20K elements). Cache the collection *and* its length; keep `forEach` off hot paths.

5.3 By DOM size and mutation rate

5.3.1 Small DOM (<1K nodes): everything fast enough except XPath; large DOM: only map-backed APIs stay flat

Below ~1K nodes, constant factors dominate and nearly every API is fast enough — the 38× small-fixture gEBTN margin is real but lands on microsecond absolute times. XPath is the sole exception: catastrophic at every size. As the document grows, the scaling law takes over: id/class/tag map-backed lookups stay flat (0.99–1.06× across 77× growth); every scan-backed call degrades ~linearly (qSA('span') 83×). At large sizes there is no selector cleverness that rescues a scan — only an index keeps you flat.

5.3.2 High mutation rate: live collections thrash invalidation — prefer static qSA or cached arrays

Live collections win under read-heavy, repeat access (§5.2.1) and lose under churn: every relevant mutation walks ancestors invalidating node-list caches,[30] and mutating `class` while iterating a class-based list invalidates the list under your feet. In mutation-dense code, pay the walk once: take a static qSA snapshot or `Array.from` a live collection into a cached array, then iterate plain memory. One scoping caveat from the sandbox must travel with this advice: document-level `querySelector` measured **1.28× faster** than the same query rooted at a near-top subtree, because the document-level class cache hit decisively on a rare class — fixture-dependent, and reversible for common classes or small deep scopes. Scope queries as a cache-miss optimization, not a reflex.

5.4 The master decision table (situation → API → measured ratio → engine reason)

All ratios measured in this investigation's sandbox (Chrome 150) unless cited; engine citations point to the mechanism chapters.

#	Situation	Use this	Measured margin	Engine reason
1	Known id, document or shadow tree	getElementById (incl. shadowRoot.getElementById)	1.18× over qS('#id'); O(1) flat 261–20K	TreeOrderedMap hash probe, interned key, no parse[28] [1]
2	Id access in a hot loop	Cache the element reference in JS	Beats every re-query path	IC field load ~1–2 instructions vs irreducible binding crossing[45]
3	First/all elements of a class, ≥1K nodes	getElementsByClassName	57× over qSA('.cls') @20K; O(1)	Class-indexed cached live list vs full scan + static list[29]
4	Elements of a tag, any size	getElementsByTagName	38×/273×/3292× by size	Tag index; qSA must visit every

#	Situation	Use this	Measured margin	Engine reason
				node
5	Known first child / sibling walk	firstElementChild / nextElementSibling	2.1× over children[0]; 2.3× over nextSibling-skip; ~2.2×/2.4× over TreeWalker/NodeIterator	Element filtering in C++; no collection/iterator allocation
6	Ancestor matching a selector	closest()	4.1× over manual loop	Native ancestor walk; no per-level JS↔C++ crossing
7	Delegation guard per bubbled event	tagName ===, then classList.contains	1.0 / 0.63 / 0.44 vs matches()	String identity < token-set lookup < selector match
8	Predicate over attributes	qSA('[data-x="1"]')	19×/12× over gEBTN('*') + JS filter	Predicate evaluated in engine; no per-node getAttribute crossing
9	Complex relational criteria	querySelector with simple key compound	Ladder spread only 3.5× @20K	Right-to-left match; key compound filters candidates[6]
10	Need an id-scoped subtree	getElementById('a').querySelectorAll('.b') + :scope prefix	#a .b never faster than .b	Same candidate set; id prefix adds ancestor walk per survivor[6] [15]
11	One-shot single hit	querySelector, not qSA(...)[0]	qSA(‘#id’) 2.3× slower even when O(1)	Early-exit vs full walk + static-list allocation[31]
12	Repeated same query, read-only	Cache one live collection (or the element)	6.1× over 100 fresh gEBCN; ~13,700× over 100 fresh qSA	Construction is the expense; liveness charges only on mutation
13	Read-only hot	for loop, live	1.23× / 1.37× /	Per-access re-

#	Situation	Use this	Measured margin	Engine reason
	iteration	collection, hoisted length	2.4× over .length-per-iter / static list / forEach	validation is a C++ round trip; callback overhead
14	Iterate while mutating	Snapshot first (Array.from, static NodeList); NodeIterator for cursor walks	Correctness: live length moves under you[7]	Invalidation per mutation; pre-remove steps keep NodeIterator honest[18]
15	Mutation-dense regime	Static snapshot or cached array, not live collections	Live lists thrash on every relevant change	Ancestor invalidation walk per mutation[30]
16	Shrinking candidates by scope	Element-rooted qSA only when document caches miss	Document qS 1.28× faster (fixture-dependent)	Document class cache resolves rare classes without walking
17	Named element access	Never window['id'] / document.forms['name']	2.8× / 8.8× slower than gEBI	Named getter + collection construction, not the id map
18	Any exotic lookup (XPath)	Rewrite as class/tag/id; mirror [attr] with a class	XPath 26× over qSA; ~70,000× over gEBCN @20K	Interpreted engine, own snapshot object, zero index
19	Small DOM (<1K), cold path	Anything except XPath	Ratios hold; absolute times trivial	Constant factors dominate below scan-cost threshold
20	Selector string reused in a loop	Hoist the string; never generate selectors dynamically	Parse caches: 256/doc (Blink), 512 (WebKit)[31] [33]	String-keyed parse caches thrash on novel strings

Rows 1–10 resolve **what you know**; rows 11–16 resolve **how you access**; rows 17–20 are the standing prohibitions and regime rules. Where two rows collide — typically 12 versus 15, cache versus churn — the mutation rate is the tiebreaker: liveness is free exactly until the DOM moves. Architecture-level questions those collisions raise (cache invalidation policy, delegation topology, component-level query ownership) belong to Chapter 6; this chapter's job ends at the call site. Write the call that keeps you on a map, off the scan path, and out of the binding layer — in that order.

6. The Library-Author Playbook

Chapter 5 answers *which call to write*; this chapter answers the prior question — *how to structure a library so the call rarely happens at all*. Twenty years of shipping code converge on one meta-principle: **the fastest element query is the one never run.** The rest is elaboration: eliminate queries by construction, route survivors to the cheapest native primitive, amortize matching through delegation or observers, discipline the collections you keep.

Library	Lookup mechanism	Event matching	Reference caching
jQuery/Sizzle	rquickExpr routing: #id→gEBI, tag→gEBTN, .class→gEBCN, else qSA; rightmost-token seeding + compiled matchers [76] [77]	Manual ancestor walk, matchesSelector per level, per-dispatch matchedSelectors memo [78]	dataPriv uid→object store; bounded LRU selector caches (cacheLength = 50) [76] [78]
Zepto	Same 3-way regex routing, ~15 lines vs Sizzle's ~2000 [79]	jQuery-compatible selector filter	None (minimalist)
React / Preact	None — refs fulfilled in commit from fiber stateNode [80]	Root listener + fiber-tree dispatch; no selector matching	useRef / callback refs hold the node [80]
Vue 3	None — vnode.el + setRef assigns the known element [81]	Patched direct listeners	Template refs at mount; raw document.querySelector flagged as defect [81] [82]
Svelte / Solid	None — compiler emits per-node closure vars [83] [84]	Direct listeners (Solid delegates a few)	Compiled-in references; bind:this / ref={el=>…} [83] [85] [84]
Lit	Scoped renderRoot.querySelector via @query getters [86]	Standard addEventListener	@query(sel, true) memoizes on instance (#2725 caveat) [86] [87] [88]
Alpine	One init walk registers directives + x-refs	Direct listeners; .self→e.target === el, .outside→el.contains(e.target) [89]	el._x_refs + memoized _x_refs_proxy [90]
htmx	qSA wrappers; id-first for preserve/OOB swaps [91]	closest() + manual-loop fallback; matches() target filters [91]	htmx-internal-data expando [91]
Stimulus	One scoped qSA +	Observer-driven —	Multimap

Library	Lookup mechanism	Event matching	Reference caching
	matches(), then MutationObserver-maintained sets [92] [93]	zero per-event matching	matchesByElement updated on mutation [93]
Turbo	Streams: gEBI(target) preferred, qSA for targets; idiomorph idMap from one qSA('[id]') [94]	Document observers + matches("a[href] ...") filters [94]	SnapshotCache LRU; permanentElementMap [94]
LWC	lwc:ref / this.refs O(1) map; scoped qS discouraged [95]	Direct listeners	Ref map measured ~2.5× over scoped qS, flat vs DOM size [95]

6.1 Rule zero: the fastest query is the one never run

6.1.1 Framework pattern: hold references from construction

The dominant pattern is reference capture at birth: if your library created the node, it already knows where the node is — storing that knowledge costs one assignment, rediscovering it costs a query forever. Svelte's compiled output is the purest form: one closure variable per node, created in `c()`, mutated in `p()`, never re-found [83]:

```
function create_fragment(ctx) {
  let h1, t0, t1;                    // references born with the nodes
  return { c() { h1 = element("h1"); t0 = text("Hello "); t1 =
text(ctx[0]); },
           p(ctx, dirty) { if (dirty & 1) set_data(t1, ctx[0]); } };
// no re-query, ever
}
```

The same idea appears at runtime in Vue's `setRef` (assigns the element the patcher already holds [81]) and React's commit-phase ref fulfillment from the fiber's `stateNode` — DOM access "without using any selector methods" [80]. The measured stakes: a held reference is ~13,700× cheaper than re-running qSA at 20K nodes, and 6.1× cheaper than even re-calling the O(1) `getElementsByClassName`, 100 reads deep (measured in this investigation's sandbox, Chrome 150).

Lit is the exception that proves the rule: it *does* query, but scopes to `renderRoot` and offers `@query(selector, /*cache*/ true)`, added in lit-element 2.4.0 "as a performance optimization … when the node being queried will not change" [86] [88]. The cache shipped a foot-gun — lit issue #2725: the memoized getter keyed on `undefined`, so a query issued before first render permanently cached `null`; the fix caches only once `hasUpdated` [87]. Generalized: **memoize query results only after the queried DOM can exist**.

6.1.2 Caching pattern catalog

When references can't be born with their nodes, these are the stores libraries actually use:

#	Pattern	Canonical user	Cost profile / justification
C1	Reference capture at construction	Svelte, Vue, React, Solid [81] [80] [83] [84]	One assignment at birth; ~13,700× over repeated qSA @20K (this investigation's sandbox, Chrome 150)
C2	Component-scoped root	Lit renderRoot, LWC template, Stimulus controller el [86] [95]	Shrinks search space; prevents cross-component collisions
C3	Lazy getter + memoization	Lit @query(sel, true) [86] [88]	One query per instance lifetime; guard on DOM existence (#2725) [87]
C4	Populate-once name→element map	Alpine _x_refs, LWC this.refs [90] [95]	O(1) reads; LWC measured ~2.5× over scoped qS, flat vs DOM growth [95]
C5	MutationObserver-maintained match set	Stimulus SelectorObserver [92] [93]	Query cost paid once per *mutation*, zero per event
C6	WeakMap<Element, State>	Framework internals, observer singletons [96]	O(1) + GC teardown; no iteration/clear; values must not strongly reference keys [96] [97]
C7	Expando / uid store	htmx htmx-internal-data, jQuery dataPriv [78] [91]	O(1); uid indirection dodges legacy circular-reference leaks [78]
C8	Bounded LRU selector cache	Sizzle createCache(), cacheLength = 50 [76] [77]	Hot selectors compiled once; bounded so selector churn can't OOM (key + " " dodges prototype collisions, #157) [76] [77]
C9	Id-keyed map	Turbo/idiomorph idMap, target= [94]	One qSA('[id]') walk turns O(n·m) morph matching into O(n)

#	Pattern	Canonical user	Cost profile / justification
			gEBI-class lookups [94]
C10	data-* attributes as lookup keys	Stimulus, htmx [91] [92] [98]	Enumerable via attributeFilter, collision-free with styling, self-describing

The generic WeakMap store (C6), for per-element metadata whose teardown you don't control [96]:

```
const stateByRoot = new WeakMap();
export function getState(el) {
  let s = stateByRoot.get(el);
  if (!s) { s = createState(); stateByRoot.set(el, s); }
  return s;                          // element GC'd ⇒ state GC'd; no
remove() bookkeeping
}
```

And the populate-once ref map (C4) — Alpine collects every `x-ref` during the *single* init walk, then memoizes the merged proxy so `$refs` never re-walks [90]:

```
magic('refs', el => {
  if (el._x_refs_proxy) return el._x_refs_proxy
  el._x_refs_proxy = mergeProxies(getArrayOfRefObject(el))
  return el._x_refs_proxy
})
```

6.2 When you must query: route like Sizzle

6.2.1 rquickExpr fast paths, rightmost-token seeding, bounded LRU

Sizzle's entry function is a routing table. Before any parsing, a bare `#id`, `tag`, or `.class` selector short-circuits to the corresponding native primitive [76]:

```
if ( nodeType !== 11 && ( match = rquickExpr.exec( selector ) ) ) {
    if ( ( m = match[ 1 ] ) ) {              // Sizzle("#ID")
        if ( nodeType === 9 ) {
            if ( ( elem = context.getElementById( m ) ) ) {
                if ( elem.id === m ) { results.push( elem ); return
results; } // name-vs-id guard
            } else { return results; }
        }
    } else if ( match[ 2 ] ) {               // Sizzle("TAG")
        push.apply( results,
```

```
context.getElementsByTagName( selector ) );
        return results;
    } else if ( ( m = match[ 3 ] ) && support.getElementsByClassName )
{  // Sizzle(".CLASS")
        push.apply( results, context.getElementsByClassName( m ) );
        return results;
    }
}
// no shortcut → qSA (if support.qsa && !rbuggyQSA.test(selector));
else compiled matcher
```

Two details matter as much as the routing. Fast paths are **guarded**: the id shortcut validates `elem.id === m` (legacy engines matched gEBI by name); the class shortcut is gated on a native-ness feature test [76]. When qSA fails, `select()` **seeds from the rightmost findable token** (ID > CLASS > TAG), filters that small seed set with a compiled matcher, and parks compiled matchers in a bounded LRU (`Expr.cacheLength = 50`) [76] [77]. The measured case for routing: gEBTN holds ~11M ops/s flat while qSA('span') collapses 38×/273×/3292× by fixture size; gEBCN beats qSA('.cls') 57× @20K (this investigation's sandbox, Chrome 150). Even the id path — gap narrowed to 1.18× by the 2026 Blink SelectorQuery rewrite — skips selector parsing entirely.

The cautionary tale: jQuery 1.4.3 (2011) routed `.find(".class")` through qSA and collapsed Twitter's scroll performance; Resig's post-mortem re-established that simple native methods beat qSA for simple selectors, and the shortcuts were backported into Sizzle [99]. **qSA is the general fallback, never the default.**

6.2.2 Scoping tricks

Element-rooted qSA has a correctness trap: `el.querySelectorAll("div span")` considers elements outside `el`. Sizzle's fix is Andrew Dupont's id-prefix hack — temporarily stamp the context with an expando id, rewrite each selector group as `#sizzle-xxx div span` (or `:scope` where supported), restore in a `finally` [76] [100]:

```
if ( context.nodeType === 1 && selector.indexOf( ">" ) !== 0 /*
not :scope-safe */ ) {
  newContext = context.getAttribute( "id" );
  newSelector = selector.replace( /^/, "#" + ( newContext || expando )
+ " " );
  if ( !newContext ) context.setAttribute( "id", expando );   //
restore in finally
}
```

Subtree narrowing as a *performance* move carries a measured caveat: `document.querySelector('.rare-class')` beat `scopeRoot.querySelector` by 1.28× in this investigation's sandbox (Chrome 150) — element-level qS cannot use the document-level class cache. Scoping pays when document caches miss *and* the subtree is small; it is an

encapsulation tool (Lit scopes for isolation [86]) and a cache-miss optimization, not a universal speedup.

6.3 Event delegation: matches() vs closest() vs manual loop

6.3.1 jQuery delegate model and the measured guard ladder

jQuery's delegated dispatch walks `event.target` up to the delegation target, calling `matchesSelector` per level, with results memoized per dispatch in `matchedSelectors[sel]` so duplicate selectors match once; selector validity is checked at bind time, failing fast instead of at event time [78]. The per-level test is where the measured guard ladder applies: `tagName === 'DIV'` at 23.97M ops/s, `classList.contains` at 0.63×, `matches('.cls')` at 0.44× (this investigation's sandbox, Chrome 150) — cheap guards precede selector evaluation:

```
root.addEventListener('click', e => {
  for (let cur = e.target; cur !== root; cur = cur.parentNode) {
    if (cur.tagName !== 'DIV') continue;            // 1.00× — string identity, no machinery
    if (!cur.classList.contains('item')) continue;  // 0.63× — token-set lookup
    if (selMemo.get(sel) ?? cur.matches(sel)) {     // 0.44× — selector engine, memoized per dispatch
      return handler.call(cur, e);
    }
  }
});
```

Honest caveat: all three exceed 10M ops/s, so the ladder matters only in genuinely hot dispatch loops — but a library's dispatch loop is by definition hot.

6.3.2 htmx closest()/matches() with manual-loop fallback; Stimulus MutationObserver Multimap

htmx exposes both idioms: native `closest()` for ancestor matching with a manual parent-loop fallback for ancient engines, `matches()` for target filters (`target:` specs), and a recursive `getClosestMatch` predicate walk where per-level semantics are needed [91]. The measured verdict collapses the choice: native `closest()` is 4.1× faster than a manual `parentNode+classList` loop on a 50-level walk, and selector complexity is nearly irrelevant (0.99×) — the engine walks in C++ with no per-level JS↔C++ crossing (this investigation's sandbox, Chrome 150). **Delete hand-rolled ancestor loops** unless you need jQuery-style per-level accumulation or target engines without `closest`.

Stimulus removes per-event matching entirely: `SelectorObserver` performs one scoped qSA + `matches()` pass, then a MutationObserver maintains `matchesByElement` incrementally [92] [93]. Matching is amortized to *mutation time*; events bind directly to already-known elements, so the per-event selector budget is zero. The cache *is* the subscription result.

6.4 Live-collection discipline and mutation-time patterns

6.4.1 Snapshot-before-mutate, cached-length loops, no gEBCN in mutation callbacks

A live HTMLCollection re-validates on access: reading `.length` per iteration costs 1.23× even in read-only loops, and mutating while iterating a live collection is a correctness bug [12]. The measured rules (Chrome 150 sandbox): cache the collection *and* its length; choose qSA's static NodeList precisely when mutation during iteration is possible; never issue gEBCN inside an unbounded mutation callback; and navigate with `firstElementChild/nextElementSibling` (2.1×/2.3× over collection indexing) instead of building a collection to read one entry:

```
// Mutate-safe: snapshot first, hoist length, iterate the static list
const list = root.querySelectorAll('.stale');   // static NodeList — safe to mutate under it
for (let i = 0, n = list.length; i < n; i++) list[i].remove();

// Hot read loop over a live collection: hoist both collection and length
const items = root.getElementsByClassName('row');
for (let i = 0, n = items.length; i < n; i++) read(items[i]);
```

6.4.2 The twelve-bullet playbook

Distilled from two decades of library source and re-anchored to this investigation's Chrome-150 measurements:

1. **Don't query — hold references.** Closure vars, `vnode.el`, fiber `stateNode`, ref callbacks [81] [80] [83]. Held reference ≈ 13,700× over repeated qSA @20K (measured).
2. **Route by selector shape:** `#id`→gEBI, `tag`→gEBTN, `.class`→gEBCN, else qSA [76]. gEBTN beats qSA('span') 3,292× @20K; gEBCN beats qSA('.cls') 57× (measured).
3. **Guard fast paths for correctness** — Sizzle validates `elem.id === m`, feature-tests gEBCN [76]. Cheap routing is worthless if it lies on edge engines.
4. **Scope for encapsulation, not assumed speed** [86]. Scoped qS measured 1.28× *slower* than document-level when the class cache hits; scope on cache misses + small subtrees only (measured, fixture-dependent).
5. **Memoize lazily, invalidate carefully.** Cache only after the DOM exists (Lit #2725's cached `null` [87]); bound selector caches (Sizzle LRU = 50 [77]).
6. **Key element metadata by WeakMap or expando** — O(1) with automatic GC teardown [96]. Holding any reference is the 6.1×-over-repeated-gEBCN win (measured).
7. **Use ids as internal lookup keys** — Turbo `target=`→gEBI, idiomorph idMap [94]. gEBI is O(1)-flat, skips selector parsing, 1.18× over qS('#id') (measured).
8. **Prefer `data-*` markers over classes for machine targeting** [92] [98]. Enumerable via `attributeFilter`; qSA('[data-x]') beats gEBTN('*')+manual filter 12–19× (measured).

9. **Seed narrow, never `qSA('*')`** — Sizzle's rightmost-token seeding minimizes candidates [76]. Selector complexity costs ≤3.5× constant factor; seed *size* is the lever (measured).
10. **Delegate at a stable ancestor; call native `closest()`.** 4.1× over manual parentNode loops; guard ladder tagName (1.00×) → classList (0.63×) → matches (0.44×); memoize per dispatch (jQuery `matchedSelectors` [78]) (measured).
11. **Let observers pay query costs once.** Stimulus's MutationObserver-maintained Multimap does zero per-event matching [93]; mutations are rare relative to events.
12. **Snapshot before mutate; hoist `.length`; navigate with element accessors.** Cached-length loop 1.23× over uncached; `firstElementChild/nextElementSibling` 2.1×/2.3× over indexing; never XPath — 26× under qSA, ~70,000× under class lookup @20K (measured).

7. Appendix: Benchmark Methodology, Sandbox, and Caveats

All numbers in chapters 3–4 come from one self-contained sandbox (`dom-bench/`). This appendix documents it completely so any reader can rerun, audit, or extend it. Environment: Chrome/150.0.7871.181 (Blink + V8), new headless mode, Linux x86_64, Puppeteer-core driving `/usr/bin/chromium`; Node.js runner, no other dependencies. Absolute ops/s values are machine-specific — ratios are the signal.

7.1 Harness design

The harness runs **in-page** (no per-call CDP overhead). Each benchmark function is wrapped so its result feeds a global XOR **sink guard** against dead-code elimination — the classic pitfall that discarded microbenchmark results are optimized away entirely [68] [69]:

```
var sink = 0;
function eatNode(n){ sink = (sink ^ (n ? (n.nodeType +
(n.nodeName.length << 4)) : 7)) | 0; }
function eatNum(x){ sink = (sink ^ (x | 0)) | 0; }
```

Every timed call passes its node or count through `eatNode/eatNum`, forcing a live value across the JS↔C++ boundary. The final `sink` value is recorded in `results.json.meta` as proof of liveness.

Timing follows a warmup/batch/median protocol:

```
var REPS = 5, SAMPLE_MS = 300, WARMUP_MS = 200;
// 1) warmup(fn, 200) for every benchmark, listed order (JIT tiering
^70^)
// 2) per sample: run fn in batches of 50 until >= 300 ms elapsed;
ops/s = n / dt
function sampleOnce(fn, minMs) {
  var t0 = performance.now(); var n = 0;
  do { for (var i = 0; i < 50; i++) fn(); n += 50; }
```

```
  while (performance.now() - t0 < minMs);
  return n / ((performance.now() - t0) / 1000);
}
// 3) 5 reps per suite run; case order rotated per rep (idx = (k +
rep*7) % N)
//    to cancel thermal/GC drift; median of samples is the reported
figure.
```

Two GC disciplines: Chromium is launched with `--js-flags=--expose-gc`, and the driver calls `window.gc()` after each fixture build and after each size suite completes — never inside a timed region, since GC pauses between samples skew distributions [71]. Finally, **read/write isolation**: no timed loop writes to the DOM, touches geometry, or reads layout properties. Interleaving reads and writes forces synchronous layout flushes and measures the renderer, not the selector engine [73]; all fixtures are built and attached *before* any timing begins.

7.2 Fixtures and launch configuration

Three size fixtures are generated procedurally (BFS, branching factor 4 → depth ~7–8), then attached to `document.body`: small = **261**, medium = **2,061**, large = **20,097** actual nodes (including injected targets). Composition: div/span/input mix, classes `.a/.b/.c` round-robin, ids on ~1% of nodes, `data-x="1"` + class `.dx` on 10%, plus a ~50-node `#scope-root` subtree with a deeply nested `.deep-target` and one `#target` id host at ~60% depth. A separate **500-deep nested-div chain** (detached; the match target sits 50 levels above the leaf) backs the `closest()`/traversal cases. Unit fixtures add interleaved text nodes for child/sibling cases.

Launch configuration (verbatim from `run.js`):

```
puppeteer.launch({
  executablePath: '/usr/bin/chromium',
  headless: 'new',
  args: ['--no-sandbox', '--disable-gpu', '--disable-dev-shm-usage',
         '--js-flags=--expose-gc']
});
```

The runner waits on `window.__done === true` (timeout 900 s, 1 s polling), then serializes `window.__results` to `results.json`.

7.3 Stability and limitations

Two independent full-suite runs agreed within **±10%** on nearly all medians (worst ~13%); reported numbers merge both runs (10 samples per measurement). Worst observed CV is **18%** (case 15 complex selectors; also case 6 and case 2 medium), traced to occasional GC pauses inside 300 ms samples — medians nonetheless stayed stable across runs; min/max in `results.json` expose those pauses. Honest limitations: (1) **single engine** — Blink/V8 only; WebKit and Gecko may differ where caches and tree scans trade off differently. (2) Headless `--disable-gpu` on shared CI-like hardware. (3) **Fixture-specific effects** — the case-6 scoped-

`querySelector` anomaly (document-level beat subtree scoping) depends on the document-level class cache locating a rare class quickly; with a common class or deeper scope the outcome can flip. (4) **jsdom numbers are not transferable** — jsdom has no engine selector caches or layout pipeline, so its ratios reflect its own pure-JS tree walk, not any real browser. (5) Case 8's `childNodes[0]` intentionally returns a text node and is not semantically equivalent.

7.4 How to reproduce and extend

File	Purpose
bench.html	Fixtures, harness (warmup/sampleOnce/runAll), all 15 case definitions; runs in-page
run.js	Puppeteer-core driver: launches headless Chromium, polls progress, writes results.json
results.json	Merged machine-readable output (medians, min/max, per-sample arrays, meta incl. UA and sink)
results_run1.json / results_run2.json	Raw per-run outputs used for the ±10% cross-run stability check
RESULTS.md	Rendered tables, key findings, stability notes for the runs above
package.json	Declares puppeteer-core; no other dependencies

To rerun: `cd dom-bench && npm i && node run.js` (progress on stderr; ~15 cases × 5 reps × ≥300 ms/sample × 3 fixture sizes, several minutes total). Run twice, saving each `results.json`, to replicate the stability check. To extend: add variants via `addCase()` inside `addSizeCases` (size-scaled) or `addUnitCases` (size-independent), always routing results through `eatNode`/`eatNum`, keeping DOM writes out of timed functions, and adding new fixtures before — never during — timing. Record UA and date with any published numbers; engine drift invalidates old ratios [67] [65].

References

1 WHATWG — DOM Standard (Living Standard): collections; DocumentOrShadowRoot/ParentNode mixins; Element.closest/matches; NodeIterator pre-remove steps; retarget algorithm (accessed 2026-08-12). https://dom.spec.whatwg.org/

2 WHATWG — HTML Standard, "Named access on the Window object" (accessed 2026-08-12). https://html.spec.whatwg.org/multipage/nav-history-apis.html#named-access-on-the-window-object

3 MDN — Document.getElementById() (accessed 2026-08-12). https://developer.mozilla.org/en-US/docs/Web/API/Document/getElementById

4 Stack Overflow — "querySelector and querySelectorAll vs getElementsByClassName and getElementById" (O(n) vs O(1) framing; live vs static; CSS.escape for IDs with colons; community benchmark table) (accessed 2026-08-12). https://stackoverflow.com/questions/14377590/queryselector-and-queryselectorall-vs-getelementsbyclassname-and-getelementbyid

5 MDN — Document.querySelectorAll() (accessed 2026-08-12). https://developer.mozilla.org/en-US/docs/Web/API/Document/querySelectorAll

6 W3C/CSSWG — CSS Selectors Level 4 (:scope; subject of a selector; "match a selector against a tree / an element" — right-to-left evaluation) (accessed 2026-08-12). https://drafts.csswg.org/selectors-4/

7 Nicholas C. Zakas — "Why is getElementsByTagName() faster than querySelectorAll()?" (DynamicNodeList vs StaticNodeList; infinite-loop demo with live lists), as reproduced in richardmyu/blog issue #1 (accessed 2026-08-12). https://github.com/richardmyu/blog/issues/1

8 MDN — TreeWalker (accessed 2026-08-12). https://developer.mozilla.org/en-US/docs/Web/API/TreeWalker

9 MDN — NodeList (accessed 2026-08-12). https://developer.mozilla.org/en-US/docs/Web/API/NodeList

10 MDN — HTMLCollection (accessed 2026-08-12). https://developer.mozilla.org/en-US/docs/Web/API/HTMLCollection

11 WHATWG — HTML Standard, "DOM" chapter (getElementsByName; document collections) (accessed 2026-08-12). https://html.spec.whatwg.org/multipage/dom.html

12 Zhihu — "getElementsByClassName 和 querySelectorAll 的正确对比" (the `.outer .inner` vs `:scope` scoped-qSA demonstration; cached-length benchmark discussion) (accessed 2026-08-12). https://zhuanlan.zhihu.com/p/156011481

13 web.dev (Jeremy Wagner, Paul Lewis) — "Reduce the scope and complexity of style calculations" (incl. Rune Lillesveen quote on selector-matching share of style calc; `.box:nth-last-child(-n+1) .title` example; elements × selectors worst case) (accessed 2026-08-12). https://web.dev/articles/reduce-the-scope-and-complexity-of-style-calculations

14 MDN — CSS performance optimization (Learn web development) (accessed 2026-08-12). https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Performance/CSS

15 MDN — Element.querySelectorAll() (accessed 2026-08-12). https://developer.mozilla.org/en-US/docs/Web/API/Element/querySelectorAll

16 MDN — Node.getRootNode() (accessed 2026-08-12). https://developer.mozilla.org/en-US/docs/Web/API/Node/getRootNode

17 Nolan Lawson — "Managing focus in the shadow DOM" (children/parentElement cannot cross shadow boundaries; getRootNode/shadowRoot needed) (accessed 2026-08-12). https://nolanlawson.com/2021/02/13/managing-focus-in-the-shadow-dom/

18 MDN — NodeIterator (accessed 2026-08-12). https://developer.mozilla.org/en-US/docs/Web/API/NodeIterator

19 MDN — Document.evaluate() (XPath / XPathResult types) (accessed 2026-08-12). https://developer.mozilla.org/en-US/docs/Web/API/Document/evaluate

20 MDN — Using shadow DOM (accessed 2026-08-12). https://developer.mozilla.org/en-US/docs/Web/API/Web_components/Using_shadow_DOM

21 WHATWG/dom issue #1422 — "Option to allow querySelector to query shadow roots" (userland recursive-walk implementation, called "nontrivial … very inefficient"; links to TreeWalker issue #1189) (accessed 2026-08-12). https://github.com/whatwg/dom/issues/1422

22 MDN — Event.composedPath() (accessed 2026-08-12). https://developer.mozilla.org/en-US/docs/Web/API/Event/composedPath

23 MDN — Event.target (accessed 2026-08-12). https://developer.mozilla.org/en-US/docs/Web/API/Event/target

24 MDN — Document.activeElement (accessed 2026-08-12). https://developer.mozilla.org/en-US/docs/Web/API/Document/activeElement

25 CSSWG issue #556 — "[cssom-view] elementFromPoint, elementsFromPoint, and caretPositionFromPoint should not return an element inside a shadow tree" (retargeting resolution) (accessed 2026-08-12). https://github.com/w3c/csswg-drafts/issues/556

26 CSSWG — CSSOM View Module (elementFromPoint/elementsFromPoint; caretPositionFromPoint with shadowRoots option) (accessed 2026-08-12). https://drafts.csswg.org/cssom-view/

27 Blink `container_node.cc` (QuerySelector/getElementsByClassName/getElementById/InvalidateNodeListCachesInAncestors) (accessed 2026-08-12). https://raw.githubusercontent.com/chromium/chromium/main/third_party/blink/renderer/core/dom/container_node.cc

28 Blink `tree_ordered_map.cc` (accessed 2026-08-12). https://raw.githubusercontent.com/chromium/chromium/main/third_party/blink/renderer/core/dom/tree_ordered_map.cc

29 Blink `node_lists_node_data.h` (live collection caching) (accessed 2026-08-12). https://raw.githubusercontent.com/chromium/chromium/main/third_party/blink/renderer/core/dom/node_lists_node_data.h

30 Blink `live_node_list_registry.h` (accessed 2026-08-12). https://raw.githubusercontent.com/chromium/chromium/main/third_party/blink/renderer/core/dom/live_node_list_registry.h

31 Blink `selector_query.cc` (SelectorQuery, SelectorQueryCache, compound fast path) (accessed 2026-08-12). https://raw.githubusercontent.com/chromium/chromium/main/third_party/blink/renderer/core/css/selector_query.cc

32 Chromium changelog 2026-06-04, "Reapplied selector query fast-path rewrites", CL 7900254 (accessed 2026-08-12). https://static.januschka.com/changelog/chromium/2026-06-04.html

33 WebKit `SelectorQuery.cpp` (accessed 2026-08-12). https://raw.githubusercontent.com/WebKit/WebKit/main/Source/WebCore/dom/SelectorQuery.cpp

34 WebKit `TreeScopeOrderedMap.cpp` (accessed 2026-08-12). https://raw.githubusercontent.com/WebKit/WebKit/main/Source/WebCore/dom/TreeScopeOrderedMap.cpp

35 WebKit trac timeline, changeset r203439 "[id=x] uses getElementById" (accessed 2026-08-12). https://trac.webkit.org/timeline?from=2016-07-20&daysback=4&authors=

36 WebKit bug 159960 / Chromium issue 627242 (YUI `[id=]` querySelector) (accessed 2026-08-12). https://bugs.webkit.org/show_bug.cgi?id=159960

37 "Overview of WebKit's CSS JIT Compiler" (Benjamin Poulain, webkit.org) (accessed 2026-08-12). https://webkit.org/blog/3271/webkit-css-selector-jit-compiler/

38 WebKit bug 140818 (CSS JIT :lang(); whole-selector slow-path fallback) (accessed 2026-08-12). https://bugs.webkit.org/show_bug.cgi?id=140818

39 Gecko `IdentifierMapEntry.h` (accessed 2026-08-12). https://raw.githubusercontent.com/mozilla/gecko-dev/master/dom/base/IdentifierMapEntry.h

40 "Why do browsers match CSS selectors from right to left?" (Boris Zbarsky, Mozilla) (accessed 2026-08-12). https://stackoverflow.com/questions/5797014/why-do-browsers-match-css-selectors-from-right-to-left

41 "Speedometer: Benchmark for Web App Responsiveness" (webkit.org) (accessed 2026-08-12). https://webkit.org/blog/3395/speedometer-benchmark-for-web-app-responsiveness/

42 WebKit bug 147980 (getElementsByClassName → HTMLCollection) (accessed 2026-08-12). https://bugs.webkit.org/show_bug.cgi?id=147980

43 "How Blink works" (haraken@chromium.org; WebIDL bindings, V8 wrappers) (accessed 2026-08-12). https://www.chromium.org/blink/

44 "Web IDL in Blink" (accessed 2026-08-12). https://www.chromium.org/blink/webidl/

45 "Fast properties in V8" (v8.dev) (accessed 2026-08-12). https://v8.dev/blog/fast-properties

46 "JavaScript engine fundamentals: Shapes and Inline Caches" (Mathias Bynens) (accessed 2026-08-12). https://mathiasbynens.be/notes/shapes-ics

47 Stack Overflow #57159219 — Performance of getElementById vs getElementsByClassName vs querySelector (accessed 2026-08-12). https://stackoverflow.com/questions/57159219/

48 MeasureThat — jQuery vs getElementById vs querySelector (accessed 2026-08-12). https://benchmarklab.azurewebsites.net/Benchmarks/Show/573/0/get-element-by-id-jquery-vs-getelementbyid-vs-querysele

49 MeasureThat — getElementById vs querySelector vs getElementsByClassName (accessed 2026-08-12). https://www.measurethat.net/Benchmarks/Show/5354/0/getelementbyid-vs-queryselector-vs-getelementsbyclassna

50 Wesley Aptekar-Cassels — getElementById vs querySelector (2021-10-29) (accessed 2026-08-12). https://blog.wesleyac.com/posts/getelementbyid-vs-queryselector

51 MeasureThat run result (accessed 2026-08-12). https://benchmarklab.azurewebsites.net/Benchmarks/ShowResult/222038

52 MeasureThat — getElementById vs querySelector (class) (accessed 2026-08-12). https://benchmarklab.azurewebsites.net/Benchmarks/Show/8896/0/getelementbyid-vs-queryselector-class

53 MeasureThat — getElementById - querySelector (Firefox Mobile 129) (accessed 2026-08-12). https://benchmarklab.azurewebsites.net/Benchmarks/Show/32162/0/getelementbyid---queryselector

54 MeasureThat — getElementById VS querySelector (simple) (accessed 2026-08-12). https://www.measurethat.net/Benchmarks/Show/16208/2/getelementbyid-vs-queryselector-simple-comparison

55 MeasureThat — getElementById vs querySelector Performance (accessed 2026-08-12). https://benchmarklab.azurewebsites.net/Benchmarks/Show/8225/0/queryselector-vs-getelementbyid-performance

56 MeasureThat — queryselector vs getelementbyid with classes and ids (accessed 2026-08-12). https://www.measurethat.net/Benchmarks/Show/31541/0/queryselector-vs-getelementbyid-with-classes-and-ids

57 MeasureThat run #12954 (Chrome 68) (accessed 2026-08-12). https://www.measurethat.net/Benchmarks/ShowResult/12954

58 MeasureThat — Get elements by class: jQuery vs qSA vs gEBCN (accessed 2026-08-12). https://www.measurethat.net/Benchmarks/Show/574/1/get-elements-by-class-jquery-vs-queryselectorall-vs-get

59 MeasureThat — TreeWalker/filter vs qSA vs NodeIterator/filter (accessed 2026-08-12). https://www.measurethat.net/Benchmarks/Show/18034/0/treewalker-filter-vs-queryselectorall-vs-nodeiterator-f

60 MeasureThat — TreeWalker for loop vs querySelectorAll(*) (accessed 2026-08-12). https://www.measurethat.net/Benchmarks/Show/18019/0/treewalker-for-loop-vs-queryselectorall

61 MeasureThat — Traverse function vs NodeIterator vs TreeWalker (accessed 2026-08-12). https://benchmarklab.azurewebsites.net/Benchmarks/Show/5581/2/traverse-function-vs-nodeiterator-vs-treewalker

62 MeasureThat — While loop vs Closest (accessed 2026-08-12). https://www.measurethat.net/Benchmarks/Show/1593/0/while-loop-vs-closest

63 MeasureThat — While loop vs Closest vs QuerySelector (accessed 2026-08-12). https://www.measurethat.net/Benchmarks/Show/18176/0/while-loop-vs-closest-vs-queryselector

64 thedotproduct.org — document.getElementById vs querySelector/querySelectorAll (2012) (accessed 2026-08-12). https://www.thedotproduct.org/posts/javascript-performance-documentgetelementbyid-versus-documentqueryselector-and-documentqueryselectorall.html

65 Android Central — Chrome Speedometer 3 optimizations (querySelector selector streamlining, 2024) (accessed 2026-08-12). https://www.androidcentral.com/apps-software/google-chrome-blows-the-competition-away-in-speedometer-3-tests

66 MeasureThat — nextSibling function vs TreeWalker (accessed 2026-08-12). https://www.measurethat.net/Benchmarks/Show/35652/1/nextsibling-function-vs-treewalker

67 Google Chromium Blog — Chrome breaks records on Speedometer 3.1 and Jetstream 3 (unified querySelector cache) (accessed 2026-08-12). https://blog.google/chromium/a-double-victory-for-web-speed-chrome-breaks-records-again-on-speedometer-31-and-jetstream-3/

68 Vyacheslav Egorov — Microbenchmarks: a fairy tale (2012) (accessed 2026-08-12). http://mrale.ph/blog/2012/12/15/microbenchmarks-fairy-tale.html

69 pkgpulse — tinybench vs mitata vs vitest bench (DCE, warmup, allocation pitfalls) (accessed 2026-08-12). https://www.pkgpulse.com/guides/tinybench-vs-mitata-vs-vitest-bench-2026

70 Mathias Bynens — Bulletproof JavaScript benchmarks (PerfPlanet 2010) (accessed 2026-08-12). https://calendar.perfplanet.com/2010/bulletproof-javascript-benchmarks/

71 Sentry blog — Frontend JavaScript Performance Testing (Benchmark.js practices) (accessed 2026-08-12). https://blog.sentry.io/frontend-javascript-performance-testing/

72 Stack Overflow #50117537 — performance.now() precision after Spectre/Meltdown (MDN quote) (accessed 2026-08-12). https://stackoverflow.com/questions/50117537/

73 Stack Insight — DOM Manipulation empirical study (layout thrashing benchmarks, methodology) (accessed 2026-08-12). https://stackinsight.dev/blog/dom-manipulation-empirical-study/

74 MeasureThat — qSA vs gEBTN + (fixed/dynamic) length + for (accessed 2026-08-12). https://benchmarklab.azurewebsites.net/Benchmarks/Show/28558/0/queryselectorall-vs-getelementsbytagname-fixeddynamic-l

75 MeasureThat — querySelectorAll vs getElementsByTagName (accessed 2026-08-12). https://www.measurethat.net/Benchmarks/Show/6066/0/queryselectorall-vs-getelementsbytagname

76 jQuery/Sizzle source — `Sizzle()` entry, `rquickExpr` shortcuts, qSA delegation, `:scope`/id-prefix hack, `select()` right-to-left seeding, `createCache()`, PR #225 note (accessed 2026-08-12). https://github.com/jquery/sizzle/blob/main/src/sizzle.js

77 "The New Sizzle" (jQuery blog, 2012) — compiled selector functions, cache, shortcut policy (accessed 2026-08-12). https://blog.jquery.com/2012/07/04/the-new-sizzle/

78 jQuery event system source — `jQuery.event.add/handlers/dispatch`, delegation walk + `matchedSelectors` memoization, `dataPriv` (accessed 2026-08-12). https://github.com/jquery/jquery/blob/main/src/event.js

79 Zepto `zepto.qsa` fast-path implementation (id/class/tag regex routing), quoted and analyzed (accessed 2026-08-12). https://www.cnblogs.com/aaronjs/p/3847964.html

80 React refs overview — DOM access without selector methods (accessed 2026-08-12). https://www.memberstack.com/blog/react-refs

81 Vue 3 `rendererTemplateRef.ts` (`setRef` — direct assignment, no querying) (accessed 2026-08-12). https://github.com/vuejs/core/blob/main/packages/runtime-core/src/rendererTemplateRef.ts

82 Vue review guidance — raw `document.querySelector` flagged as high-priority defect (accessed 2026-08-12). https://github.com/affaan-m/everything-claude-code/blob/main/agents/vue-reviewer.md

83 Svelte compiled output walkthrough (direct node variables) (accessed 2026-08-12). https://lihautan.com/compile-svelte-in-your-head-part-1

84 Solid.js fine-grained reactivity — compiled direct DOM references, refs (accessed 2026-08-12). https://skarif.dev/blog/2024/solidjs-fine-grained-reactivity-without-virtual-dom

85 Svelte component guidance — avoid querySelector, prefer `bind:this`/actions (accessed 2026-08-12). https://geoffrich.net/posts/clean-component-tips/

86 Lit `@query` decorator source — `renderRoot.querySelector` getter + cache flag (accessed 2026-08-12). https://github.com/lit/lit/blob/main/packages/reactive-element/src/decorators/query.ts

87 lit issue #2725 — cached `null` bug in `@query(cache)` (accessed 2026-08-12). https://github.com/lit/lit/issues/2725

88 lit-element CHANGELOG 2.4.0 — `cache` argument added to `@query` (PR #1013) (accessed 2026-08-12). https://github.com/lit/lit/blob/main/packages/lit-element/CHANGELOG.md

89 Alpine `on` utility — direct listeners, `.self`/`.outside` modifiers (accessed 2026-08-12). https://github.com/alpinejs/alpine/blob/main/packages/alpinejs/src/utils/on.js

90 Alpine `$refs` magic — `_x_refs` / `_x_refs_proxy` memoization (accessed 2026-08-12). https://github.com/alpinejs/alpine/blob/main/packages/alpinejs/src/magics/$refs.js

91 htmx 1.x source — `matches()`, `closest()` + fallback loop, `getClosestMatch`, `querySelectorAllExt`, `findElementsToProcess`, `getInternalData`, `handlePreservedElements`/`oobSwap` id lookups (accessed 2026-08-12). https://github.com/bigskysoftware/htmx

92 Stimulus `AttributeObserver` — `hasAttribute` matching + scoped qSA (accessed 2026-08-12). https://github.com/hotwired/stimulus/blob/main/src/mutation-observers/attribute_observer.ts

93 Stimulus `SelectorObserver` — `element.matches`, `matchElementsInTree` qSA, `Multimap matchesByElement` (accessed 2026-08-12). https://github.com/hotwired/stimulus/blob/main/src/mutation-observers/selector_observer.ts

94 Turbo source — `StreamActions`, `targetElementsById` (`getElementById`) vs `targetElementsByQuery` (qSA), idiomorph `idMap` from `querySelectorAll("[id]")` (accessed 2026-08-12). https://github.com/hotwired/turbo

95 Salesforce LWC Template Refs — "without the need to use any querySelector" (accessed 2026-08-12). https://developer.salesforce.com/blogs/2023/01/lwc-enhancements-for-developers-learn-moar-spring-23

96 WeakMap observer-singleton + element-metadata caching patterns (accessed 2026-08-12). https://yceffort.kr/en/2026/01/intersection-observer-singleton-weakmap

97 WeakMap usage/leak caveats (accessed 2026-08-12). https://buglyst.com/learn/guides/javascript-weakmap-memory-leak-debug

98 htmx v4 docs/migration notes — narrow selectors, `data-*` markers (accessed 2026-08-12). https://vibetuner.alltuner.com/htmx-migration/

99 John Resig, "Learning from Twitter" — jQuery 1.4.3 qSA regression, Sizzle shortcut backport (accessed 2026-08-12). https://johnresig.com/blog/learning-from-twitter/

100 Element-rooted qSA scoping bug & id-prefix workaround (Andrew Dupont technique) (accessed 2026-08-12). https://www.cnblogs.com/dolphinX/p/3354318.html